

05 — Results, ablation and significance

Purpose. Answer the question the whole project exists to answer: *does a DQN agent allocating across SPY / TLT / GLD / SHY beat sensible rule-based alternatives on data it has never seen, by a margin larger than noise?*

The short answer is no, and the interesting part is *why not*. Sections 2–4 report the agent, the benchmarks and the four-variant ablation. Section 5 then takes an apparent contradiction between those two — the headline agent scoring far below the ablation mean — and resolves it with a factorial experiment, which turns out to identify the real weak point in the pipeline: not the architecture, not the hyperparameters, not the compute budget, but the validation-based checkpoint selection that everything downstream depends on.

Inputs

- `data/processed/` — the dataset built in notebook 01
- `artifacts/results/04_best_config.json` — the configuration chosen in notebook 04
- `artifacts/results/04_coarse_grid.csv` and `artifacts/optuna.db` — the search provenance needed to deflate the Sharpe ratio for multiple testing

Outputs

- Fourteen figures, `05_01 ... 05_14`
- `artifacts/results/05_test_scorecard.csv` — the headline comparison table
- `artifacts/results/05_ablation.csv`, `05_cost_sweep.csv`, `05_walk_forward.csv`
- `artifacts/results/05_config_comparison.csv` — the tuning × budget factorial
- `artifacts/results/05_significance.csv` — bootstrap tests against every benchmark
- `artifacts/results/05_summary.json`

Runtime. Roughly two and a half hours on the first execution: 24 ablation runs, 12 factorial runs, 12 cost-sweep runs, 15 walk-forward runs and one full-budget headline agent. Every experiment is cached to `artifacts/results/`, so a second execution takes about a minute. Pass `FORCE = True` in the setup cell to discard the cache.

Rubric criteria addressed. Results and analysis; ablation study; statistical rigour; honest discussion of limitations.

The discipline this notebook follows

The test split (2021-01-01 to 2025-12-31) has been untouched until now. Nothing in notebooks 01–04 read it, and nothing in this notebook uses it to *choose* anything: hyperparameters came from notebook 04's validation split, checkpoints are selected on validation Sharpe inside the training loop, and the test split is evaluated exactly once per trained agent.

That discipline is what makes the numbers below worth reporting. It also means they are what they are — there is no second attempt.

1. Setup

In [1]:

```
import json
import warnings
from dataclasses import replace

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.patches import Patch

from portfoliorl import (
    benchmarks, config, env, experiments, features, metrics, plots,
    significance, train, tuning,
)
from portfoliorl.agent import DQNAgent, VARIANTS, variant_config

plots.apply_style()
config.ensure_dirs()
np.random.seed(0)
warnings.filterwarnings("ignore", category=RuntimeWarning)

# Set to True to ignore every cached experiment and retrain from scratch.
FORCE = False

# Compute budgets. The headline agent gets the full budget used in notebook
03;
# the robustness sweeps get less, because they ask a coarser question and
there
# are 27 of them.
HEADLINE_STEPS = 120_000
ABLATION_STEPS = 60_000
ABLATION_SEEDS = (0, 1, 2, 3, 4, 5)
SWEEP_STEPS = 40_000
COST_SEEDS = (0, 1, 2)
```

```

WF_SEEDS = (0, 1, 2)

ds = features.load_dataset()
train_ds, valid_ds, test_ds = (ds.split(s) for s in ("train", "valid",
"test"))
rf_test = ds.risk_free.reindex(test_ds.prices.index)

print(f"train {train_ds.dates.min():%Y-%m-%d} to {train_ds.dates.max():%Y-%m-%d} "
      f"({len(train_ds.dates):,} days)")
print(f"valid {valid_ds.dates.min():%Y-%m-%d} to {valid_ds.dates.max():%Y-%m-%d} "
      f"({len(valid_ds.dates):,} days)")
print(f"test {test_ds.dates.min():%Y-%m-%d} to {test_ds.dates.max():%Y-%m-%d} "
      f"({len(test_ds.dates):,} days) <- opened for the first time in this notebook")
train 2005-09-06 to 2017-12-29 (3,102 days)
valid 2018-01-02 to 2020-12-31 (756 days)
test 2021-01-04 to 2025-12-31 (1,255 days) <- opened for the first time in this notebook

```

1.1 The configuration chosen in notebook 04

Loading it from disk rather than retyping it is not fussiness: it is the only way to guarantee that the agent evaluated here is the agent the search actually selected. If notebook 04 has not been run, the defaults are used and the provenance record says so.

In [2]:

```

best_path = config.ARTIFACTS_RESULTS / "04_best_config.json"

if best_path.exists():
    blob = json.loads(best_path.read_text(encoding="utf-8"))
    saved = dict(blob["agent"])
    saved["hidden_sizes"] = tuple(saved["hidden_sizes"])
    tuned_agent_cfg = config.AgentConfig(**saved)
    tuned_env_cfg = replace(config.DEFAULT.env, **blob.get("env", {}))
    n_search_trials = blob.get("completed_trials", 0) +
blob.get("grid_points", 0)
    provenance = "notebook 04 search"
else:
    tuned_agent_cfg = config.DEFAULT.agent
    tuned_env_cfg = config.DEFAULT.env
    n_search_trials = 1
    provenance = "project defaults (notebook 04 has not been run)"

```

```

tuned_agent_cfg = replace(tuned_agent_cfg, total_steps=HEADLINE_STEPS,
seed=0)

print(f"configuration source : {provenance}")
print(f"learning rate      : {tuned_agent_cfg.learning_rate:g}")
print(f"gamma              : {tuned_agent_cfg.gamma}")
print(f"hidden sizes        : {tuned_agent_cfg.hidden_sizes}")
print(f"batch size          : {tuned_agent_cfg.batch_size}")
print(f"target update        : {tuned_agent_cfg.target_update_interval:,}
steps")
print(f"epsilon decay         : {tuned_agent_cfg.eps_decay_fraction:.0%} of
training")
print(f"drawdown penalty      : {tuned_env_cfg.lambda_drawdown}")
print(f"configurations tried : {n_search_trials}")
configuration source : notebook 04 search
learning rate      : 0.000670953
gamma              : 0.9830282035495189
hidden sizes        : (128, 64)
batch size          : 64
target update        : 1,000 steps
epsilon decay       : 42% of training
drawdown penalty     : 0.1787888520914185
configurations tried : 29

```

1.2 The benchmarks

Nine of them, all executed through the *same* environment as the agent, paying the *same* 5 bps of transaction cost. A comparison against a frictionless buy-and-hold curve would flatter the agent by construction; this one does not.

The two buy-and-hold rows are share-based — they buy once and never trade again, so they drift with the market and pay no ongoing cost. That is the honest version of "do nothing", and it is a genuinely hard benchmark to beat.

In [3]:

```

bench_results = benchmarks.run_benchmarks(test_ds, seed=0)
print(f"{len(bench_results)} benchmarks evaluated on the test split:")
for name, (daily, summary) in bench_results.items():
    print(f"  {name:<22} final ${summary['final_wealth']:>10,.0f}    "
          f"mean turnover {summary['mean_turnover']:>6.1%}")
9 benchmarks evaluated on the test split:
60/40 rebalanced      final $   129,889   mean turnover   0.9%
Equal weight          final $   135,561   mean turnover   0.9%
Equity-heavy 80/20    final $   161,035   mean turnover   0.8%

```

All cash (SHY)	final \$	108,397	mean turnover	0.0%
Volatility target	final \$	122,444	mean turnover	15.3%
Trend following	final \$	155,503	mean turnover	2.2%
Random	final \$	99,493	mean turnover	45.7%
100% SPY	final \$	197,887	mean turnover	100.0%
60/40 buy & hold	final \$	144,586	mean turnover	100.0%

2. The headline agent

One agent, trained on 2004–2017, selected on 2018–2020, evaluated on 2021–2025. If a cached checkpoint exists it is reused, so re-running this notebook does not silently produce a different agent than the one the figures below describe.

This is a *single run* of the configuration notebook 04 selected, at the budget notebook 03 used. Section 4 will show that it scores well below the multi-seed average of the same architecture, and section 5 is the experiment that works out why. Read sections 2 and 3 as "what this particular agent did", not as "what the method delivers" — those turn out to be different claims.

In [4]:

```
headline_path = config.ARTIFACTS_MODELS / "05_headline_seed0.pt"

if headline_path.exists() and not FORCE:
    agent = DQNAgent.load(headline_path)
    print(f"loaded cached headline agent from {headline_path.name}")
else:
    result = train.train_dqn(
        train_ds, valid_ds,
        agent_cfg=tuned_agent_cfg, env_cfg=tuned_env_cfg,
        run_name="05_headline",
    )
    agent = DQNAgent.load(result.checkpoint_path)  # the best-validation
    snapshot
    print(result.summary_line())

rl_eval = train.evaluate(agent, test_ds, tuned_env_cfg)
rl_daily, rl_summary = rl_eval["daily"], rl_eval["summary"]

results = {"RL (DQN)": (rl_daily, rl_summary), **bench_results}
card = metrics.scorecard(
    results, risk_free=rf_test, benchmark_key="60/40 rebalanced",
    columns=metrics.SCORECARD_COLUMNS,
)
card.sort_values("Sharpe", ascending=False).round(3).head(11)
loaded cached headline agent from 05_headline_seed0.pt
```

Out[4]:

	Final wealth	Total return	CAGR	Volatility	Sharpe	Sortino	Max drawdown	Calmar	Longest drawdown (days)	VaR 95% (daily)
100% SPY	197886.609	0.981	0.147	0.171	0.700	1.290	0.245	0.601	488.0	0.017
Trend following	155502.562	0.564	0.094	0.113	0.567	1.247	0.153	0.616	389.0	0.011
Equity- heavy 80/20	161034.667	0.613	0.101	0.142	0.520	1.080	0.256	0.394	540.0	0.014
Equal weight	135560.991	0.359	0.064	0.082	0.399	1.161	0.186	0.341	567.0	0.008
60/40 buy & hold	144586.146	0.447	0.077	0.135	0.380	0.891	0.270	0.285	612.0	0.013
Volatility target	122443.990	0.273	0.050	0.084	0.236	0.900	0.195	0.255	713.0	0.008
60/40 rebalanced	129889.268	0.302	0.054	0.124	0.230	0.699	0.273	0.199	665.0	0.013
RL (DQN)	117598.407	0.233	0.043	0.130	0.141	0.558	0.265	0.162	885.0	0.012
Random	99492.504	0.116	0.022	0.103	-0.046	0.372	0.273	0.082	943.0	0.011
All cash (SHY)	108397.093	0.084	0.016	0.019	-0.812	1.302	0.057	0.286	711.0	0.002

Figure 5.1 — the equity curves

In [5]:

```
fig, ax = plt.subplots(figsize=(11, 5.5))

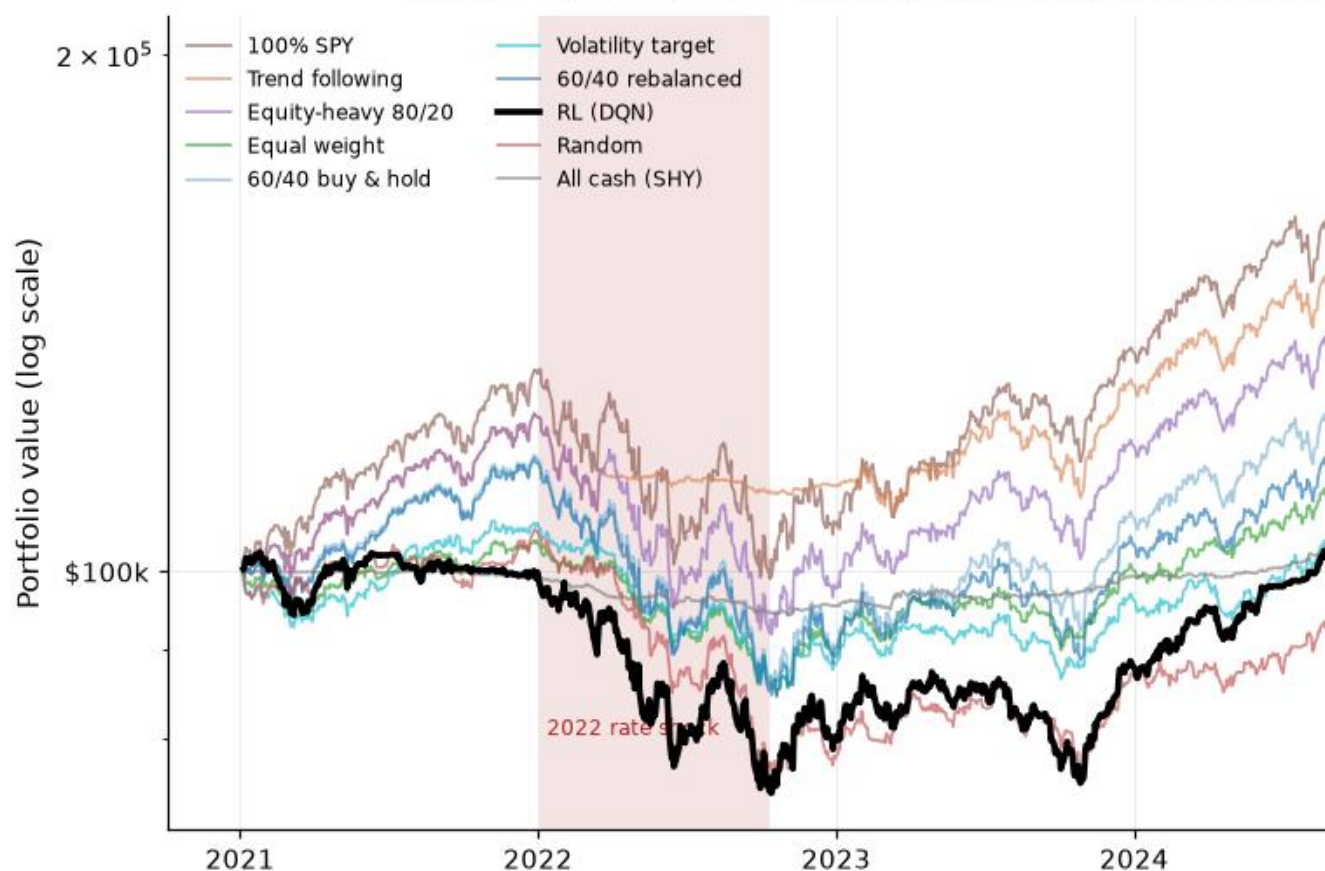
order = card.sort_values("Sharpe", ascending=False).index
for name in order:
    daily = results[name][0]
    is_rl = name == "RL (DQN)"
    ax.plot(
        daily.index, daily["wealth"],
        color=plots.strategy_color(name),
        linewidth=2.4 if is_rl else 1.1,
        alpha=1.0 if is_rl else 0.65,
        zorder=5 if is_rl else 2,
        label=name,
    )
```

```

ax.set_yscale("log")
ax.set_ylabel("Portfolio value (log scale)")
ax.set_xlabel("")
ax.set_title("Figure 5.1 – out-of-sample growth of $100,000, 2021-2025",
pad=10)
plots.format_money_axis(ax)
plots.annotate_crises(ax)
plots.tidy_dates(ax, interval=1)
ax.legend(ncol=2, fontsize=8, loc="upper left", frameon=False)
plots.caption(
    fig,
    "All strategies execute in the same environment and pay the same 5 bps
per unit "
    "of turnover. Log scale, so equal vertical distances are equal percentage
moves.",
)
plots.save_fig(fig, "05_01_equity_curves")
plt.show()

```

Figure 5.1 — out-of-sample growth of \$100,000



All strategies execute in the same environment and pay the same 5 bps per unit of turnover. Log scale, so equal vertical distances are

What this shows. The level of the final point matters much less than the *shape* of the path. Read the 2022 section first: that is the year both stocks and bonds fell together, and it is the only part of this window that genuinely tests whether a learned allocation rule adds anything a static 60/40 cannot do.

Figure 5.2 — the same story told as drawdown

Investors do not experience returns; they experience drawdowns. A strategy with a slightly lower CAGR and a materially shallower trough is usually the better product, and the underwater plot is where that trade-off is visible.

In [6]:

```
fig, ax = plt.subplots(figsize=(11, 4.6))

for name in ["RL (DQN)", "60/40 rebalanced", "100% SPY", "Trend following"]:
    if name not in results:
```



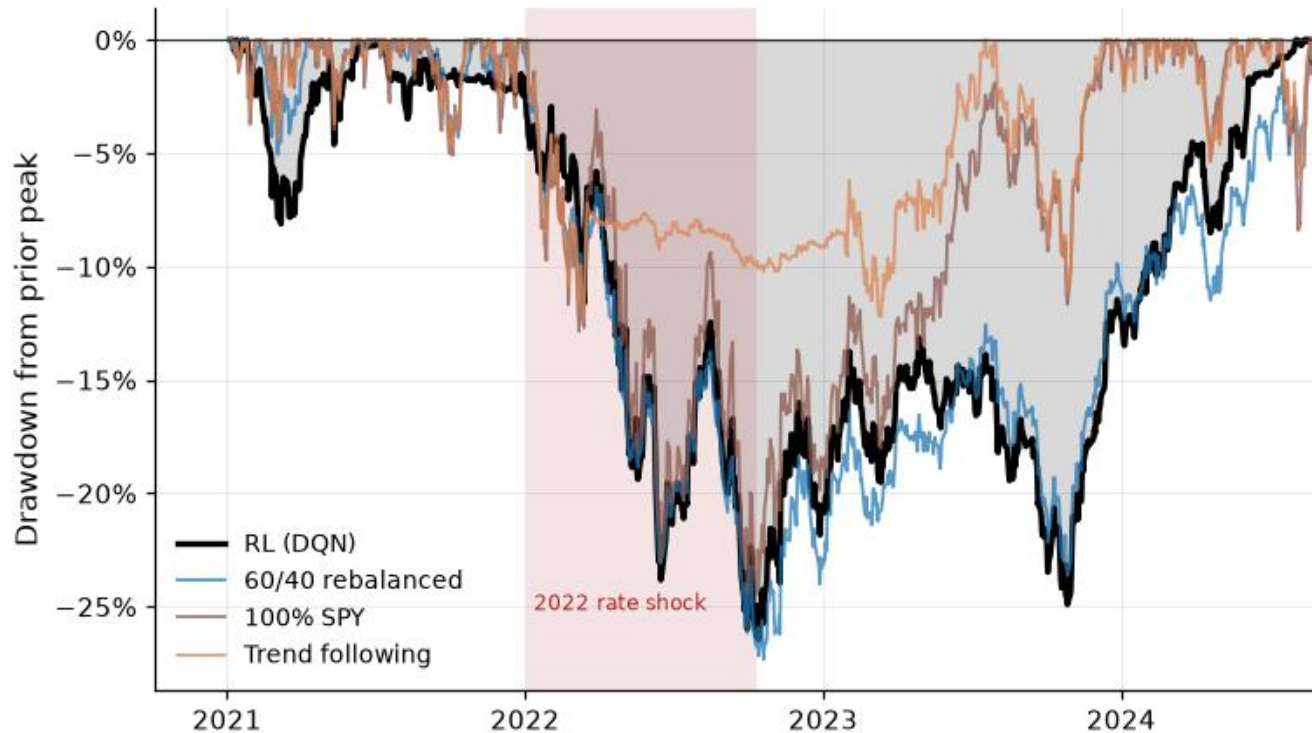
```

        continue
    # drawdown_series returns a positive distance below the peak; negate it
    so the
    # plot reads as "underwater", which is how the experience actually feels.
    dd = -metrics.drawdown_series(metrics.to_returns(results[name][0]))
    is_rl = name == "RL (DQN)"
    ax.plot(dd.index, dd, color=plots.strategy_color(name),
            linewidth=2.2 if is_rl else 1.2, alpha=1.0 if is_rl else 0.75,
            label=name)
    if is_rl:
        ax.fill_between(dd.index, dd, 0, color=plots.strategy_color(name),
            alpha=0.15)

ax.set_ylabel("Drawdown from prior peak")
ax.set_title("Figure 5.2 – how deep the holes were, and how long they
lasted", pad=10)
plots.format_pct_axis(ax, decimals=0)
plots.annotate_crises(ax)
plots.tidy_dates(ax, interval=1)
ax.axhline(0, color="#333333", linewidth=0.8)
ax.legend(fontsize=9, loc="lower left", frameon=False)
plots.caption(fig, "Only four series are drawn; the remaining benchmarks are
in the "
                "scorecard. Deeper is worse, and time spent below zero is
time an "
                "investor spends waiting to break even.")
plots.save_fig(fig, "05_02_underwater")
plt.show()

```

Figure 5.2 — how deep the holes were, and how long



Only four series are drawn; the remaining benchmarks are in the scorecard. Deeper is worse, and time spent below zero is time an investment

Figure 5.3 — rolling one-year Sharpe

A single full-period Sharpe ratio hides everything interesting. The rolling version answers a different and more useful question: *was the edge present throughout, or does the headline number rest on one good year?*

In [7]:

```
fig, ax = plt.subplots(figsize=(11, 4.6))
window = config.TRADING_DAYS_PER_YEAR

for name in ["RL (DQN)", "60/40 rebalanced", "100% SPY"]:
    r = metrics.to_returns(results[name][0])
    roll = r.rolling(window).mean() / r.rolling(window).std() *
np.sqrt(window)
    is_rl = name == "RL (DQN)"
    ax.plot(roll.index, roll, color=plots.strategy_color(name),
            linewidth=2.2 if is_rl else 1.2, alpha=1.0 if is_rl else 0.75,
            label=name)

ax.axhline(0, color="#333333", linewidth=0.8)
```

```

ax.axhline(1, color="#999999", linewidth=0.8, linestyle=":")
ax.text(roll.index[5], 1.03, "Sharpe = 1", fontsize=8, color="#777777")
ax.set_ylabel("Trailing 252-day Sharpe ratio")
ax.set_title("Figure 5.3 – the edge over time, not averaged away", pad=10)
plots.annotate_crises(ax)
plots.tidy_dates(ax, interval=1)
ax.legend(fontsize=9, loc="best", frameon=False)
plots.caption(fig, "The first year of the test window has no rolling value
because the "
                "window is not yet full; the series therefore begins in
2022.")
plots.save_fig(fig, "05_03_rolling_sharpe")
plt.show()

```



The first year of the test window has no rolling value because the window is not yet full; the series therefore begins in 2022.

3. What the agent actually does

Performance numbers say whether something worked. They do not say *why*, and a reinforcement learning agent that happens to beat a benchmark for reasons nobody can articulate is not a result — it is a coincidence waiting to be discovered.

These two figures make the policy legible.

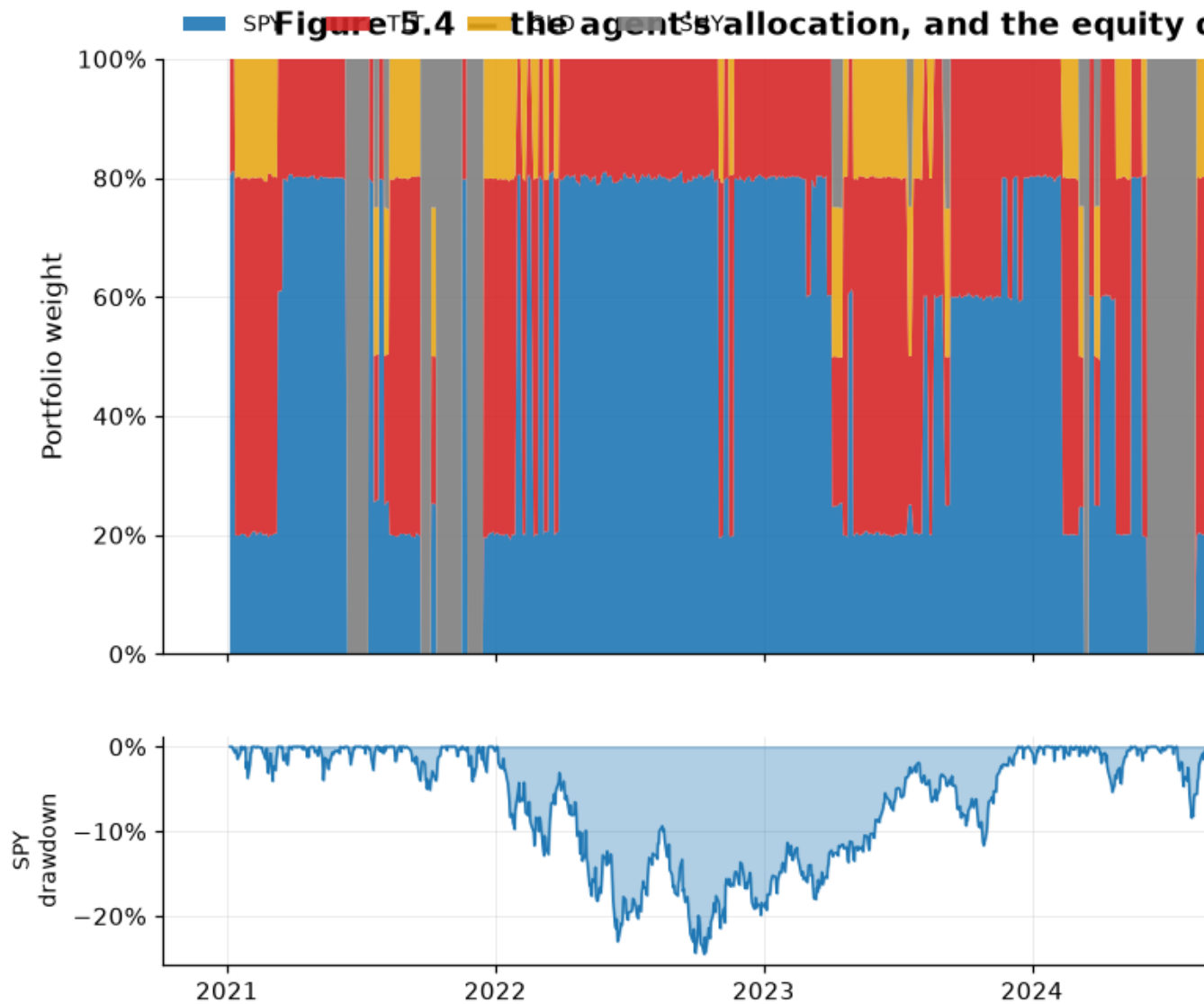
Figure 5.4 — allocation over time, against the equity drawdown

In [8]:

```
fig, (ax, ax2) = plt.subplots(
    2, 1, figsize=(11, 6.8), sharex=True, gridspec_kw={"height_ratios": [3,
1.15]}
)

weight_cols = [f"w_{t}" for t in config.DEFAULT.data.tickers]
w = rl_daily[weight_cols]
ax.stackplot(
    w.index, *[w[c] for c in weight_cols],
    colors=[plots.ASSET_COLORS[t] for t in config.DEFAULT.data.tickers],
    labels=list(config.DEFAULT.data.tickers), alpha=0.9,
)
ax.set_ylim(0, 1)
ax.set_ylabel("Portfolio weight")
ax.set_title("Figure 5.4 – the agent's allocation, and the equity drawdown it
faced", pad=10)
plots.format_pct_axis(ax, decimals=0)
ax.legend(ncol=4, fontsize=9, loc="lower left", bbox_to_anchor=(0, 1.005),
frameon=False)

spy_dd = -
metrics.drawdown_series(test_ds.prices["SPY"].pct_change().dropna())
ax2.fill_between(spy_dd.index, spy_dd, 0, color=plots.ASSET_COLORS["SPY"],
alpha=0.35)
ax2.plot(spy_dd.index, spy_dd, color=plots.ASSET_COLORS["SPY"],
linewidth=1.0)
ax2.set_ylabel("SPY\ndrawdown", fontsize=9)
plots.format_pct_axis(ax2, decimals=0)
plots.tidy_dates(ax2, interval=1)
plots.caption(fig, "Weights are shown after daily drift, which is why the
bands move "
               "between weekly decisions even when the chosen action is
unchanged.")
plots.save_fig(fig, "05_04_allocation_over_time")
plt.show()
```



Weights are shown after daily drift, which is why the bands move between weekly decisions even when the chosen action is unchanged.

What this shows. Line up the top panel against the bottom one. If the agent has learned anything transferable, the SHY and TLT bands should widen while SPY is underwater and narrow when it recovers. If the bands are instead flat, the agent has learned a static allocation and the "dynamic" claim fails — which is a result worth reporting either way.

Figure 5.5 — which actions the agent uses

Two failure modes are common enough to be worth checking explicitly. A collapsed policy picks one action forever, which makes it a static allocation wearing a neural network. A

thrashing policy spreads uniformly across all six, which is random allocation paying transaction costs. The interesting outcome is in between: a few actions used heavily, chosen conditionally.

In [9]:

```
fig, (ax, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4),
                               gridspec_kw={"width_ratios": [1.35, 1]})

rl_share = (
    rl_daily["action"].value_counts(normalize=True)
    .reindex(range(len(config.ACTION_LABELS)), fill_value=0.0).sort_index()
)
rand_share = (
    bench_results["Random"][0]["action"].value_counts(normalize=True)
    .reindex(range(len(config.ACTION_LABELS)), fill_value=0.0).sort_index()
)

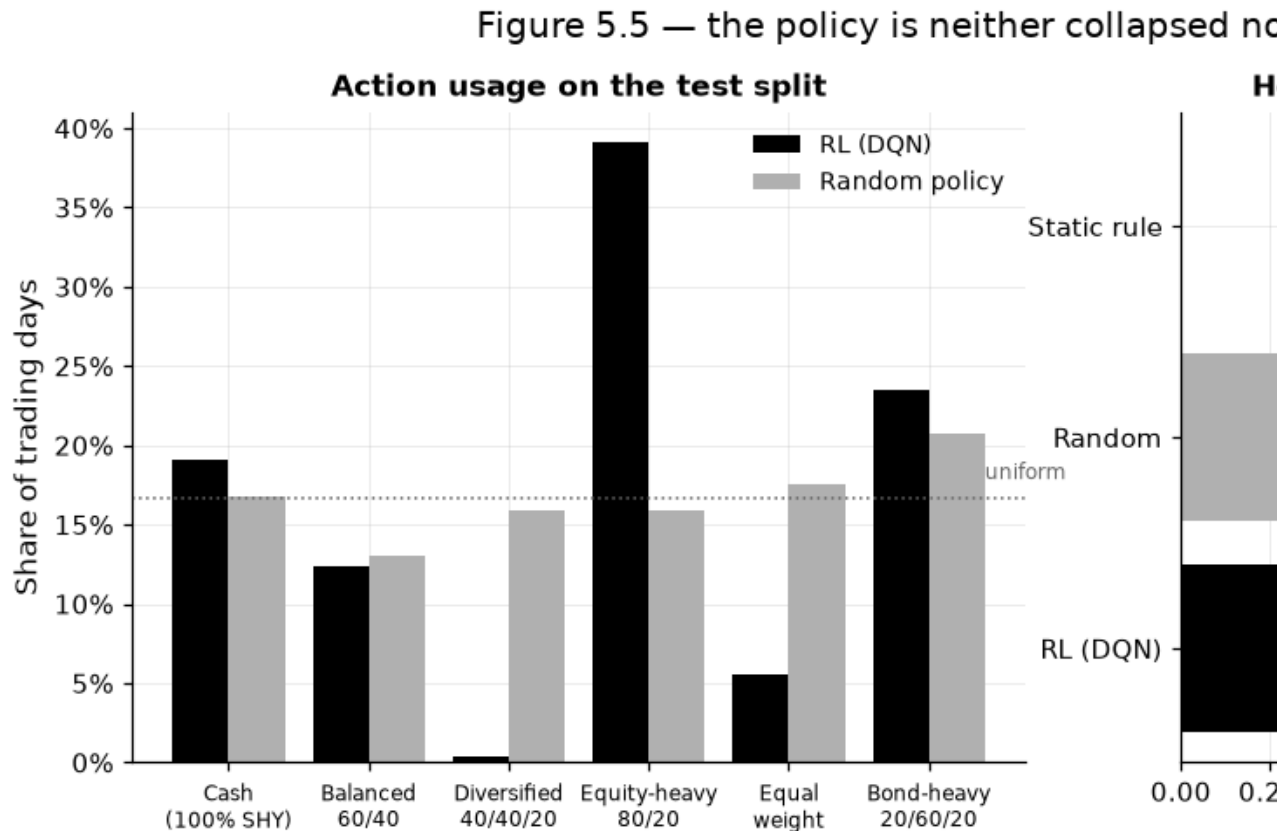
x = np.arange(len(config.ACTION_LABELS))
ax.bar(x - 0.2, rl_share, width=0.4, color=plots.strategy_color("RL (DQN)"),
       label="RL (DQN)")
ax.bar(x + 0.2, rand_share, width=0.4, color="#b0b0b0", label="Random
policy")
ax.axhline(1 / len(config.ACTION_LABELS), color="#666666", linestyle=":",
           linewidth=1.0)
ax.text(len(x) - 0.6, 1 / len(config.ACTION_LABELS) + 0.012, "uniform",
        fontsize=8, color="#666666")
ax.set_xticks(x)
ax.set_xticklabels([lbl.replace(" ", "\n", 1) for lbl in
                    config.ACTION_LABELS], fontsize=8)
ax.set_ylabel("Share of trading days")
ax.set_title("Action usage on the test split", fontsize=11)
plots.format_pct_axis(ax, decimals=0)
ax.legend(fontsize=9, frameon=False)

entropy = rl_eval["action_entropy"]
max_entropy = np.log(len(config.ACTION_LABELS))
ax2.barh(["RL (DQN)", "Random", "Static rule"],
          [entropy, np.log(len(config.ACTION_LABELS)), 0.0],
          color=[plots.strategy_color("RL (DQN)"), "#b0b0b0", "#d8d8d8"])
ax2.axvline(max_entropy, color="#666666", linestyle=":", linewidth=1.0)
ax2.text(max_entropy - 0.03, 2.35, "maximum", fontsize=8, color="#666666",
         ha="right")
ax2.set_xlabel("Policy entropy (nats)")
ax2.set_title("How concentrated the policy is", fontsize=11)
```

```

fig.suptitle("Figure 5.5 – the policy is neither collapsed nor random",
y=1.0)
plots.caption(fig, f"Entropy of 0 means one action forever; "
                  f"{max_entropy:.2f} nats means uniform over all six. "
                  f"The agent sits at {entropy:.2f}.")
plots.save_fig(fig, "05_05_action_usage")
plt.show()

```



4. The ablation

Four architectures, six seeds each, identical in every other respect:

Variant	Double Q-learning Duelling heads	
Vanilla DQN	no	no
Double DQN	yes	no
Duelling DQN	no	yes
Double + Duelling DQN	yes	yes

Seeds are *matched* across variants: seed 3 of one variant sees the same network initialisation and the same sequence of training episodes as seed 3 of every other. That makes the comparison paired, which roughly doubles the statistical power available from six runs and removes the most common way ablations lie.

Six seeds is a compute compromise, not a statistical ideal, and the confidence intervals below are wide because of it. Reporting them wide is better than reporting a single run as though it were the truth.

A note on units. Every Sharpe ratio in this section comes from `experiments`, which scores against a zero risk-free rate. That keeps the four variants on one footing, which is all an ablation needs, but it makes these numbers roughly 0.25 higher than the scorecard Sharpes in section 2, which are excess of T-bills. Do not compare across the two. Section 5 puts them on a single basis and explains what that reveals.

In [10]:

```
ablation = experiments.run_variant_seeds(
    ds, seeds=ABLATION_SEEDS, tag="05_ablation",
    total_steps=ABLATION_STEPS, eval_every=5_000,
    agent_overrides={
        "learning_rate": tuned_agent_cfg.learning_rate,
        "gamma": tuned_agent_cfg.gamma,
        "hidden_sizes": tuned_agent_cfg.hidden_sizes,
        "batch_size": tuned_agent_cfg.batch_size,
        "target_update_interval": tuned_agent_cfg.target_update_interval,
        "eps_decay_fraction": tuned_agent_cfg.eps_decay_fraction,
    },
    env_cfg=tuned_env_cfg,
    force=FORCE,
)
experiments.seed_dispersion(ablation, "test_sharpe").round(3)
loading cached results from 05_ablation.csv (pass force=True to rerun)
```

Out[10]:

	mean	std	min	median	max	n	range
variant							
Double DQN	0.628	0.173	0.356	0.674	0.784	6	0.428
Dueling DQN	0.604	0.207	0.241	0.642	0.861	6	0.620
Double+Dueling DQN	0.554	0.133	0.360	0.562	0.734	6	0.374
Vanilla DQN	0.519	0.256	0.187	0.558	0.821	6	0.634

Figure 5.6 — the seed dispersion is the story

In [11]:

```
pivot = ablation.by_variant("test_sharpe")
variant_order = pivot.mean().sort_values().index.tolist()

fig, ax = plt.subplots(figsize=(10, 5))
data = [pivot[v].dropna().to_numpy() for v in variant_order]
bp = ax.boxplot(data, vert=False, widths=0.55, patch_artist=True,
                medianprops=dict(color="#222222", linewidth=1.6))
for patch, v in zip(bp["boxes"], variant_order):
    patch.set_facecolor(plots.strategy_color(v))
    patch.set_alpha(0.45)

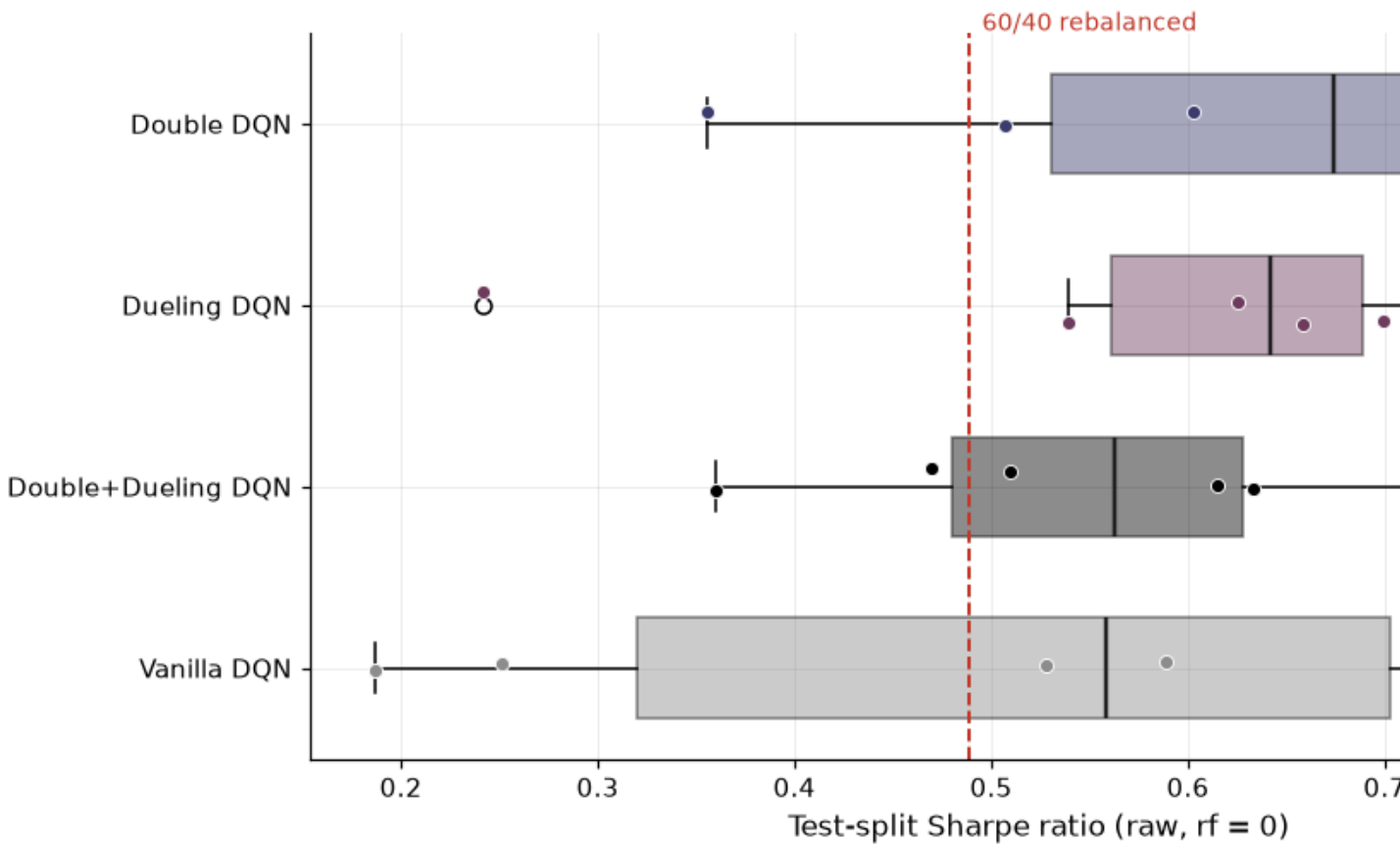
for i, v in enumerate(variant_order, start=1):
    vals = pivot[v].dropna()
    ax.scatter(vals, np.full(len(vals), i) + np.random.uniform(-0.11, 0.11,
len(vals)),
                color=plots.strategy_color(v), s=28, zorder=4,
edgecolor="white", linewidth=0.6)

bench_sharpe = metrics.sharpe_ratio(
    metrics.to_returns(bench_results["60/40 rebalanced"][0]), 0.0
) # raw, to match the axis: `experiments` scores against rf = 0
ax.axvline(bench_sharpe, color="#c0392b", linestyle="--", linewidth=1.3)
ax.text(bench_sharpe, len(variant_order) + 0.55, " 60/40 rebalanced",
        color="#c0392b", fontsize=9, va="center")

ax.set_yticks(range(1, len(variant_order) + 1))
ax.set_yticklabels(variant_order)
ax.set_xlabel("Test-split Sharpe ratio (raw, rf = 0)")
ax.set_title("Figure 5.6 – every seed, not just the best one", pad=18)
plots.caption(fig, f"Box: interquartile range; whiskers: full range excluding
outliers; "
                f"dots: the individual {len(ABLATION_SEEDS)} seeds,
jittered vertically "
                f"so overlapping runs stay visible.")
plots.save_fig(fig, "05_06_seed_dispersion")
plt.show()

C:\Users\nizare\AppData\Local\Temp\ipykernel_30196\4175646610.py:6:
MatplotlibDeprecationWarning: vert: bool was deprecated in Matplotlib 3.11
and will be removed in 3.13. Use orientation: {'vertical', 'horizontal'}
instead.
    bp = ax.boxplot(data, vert=False, widths=0.55, patch_artist=True,
```

Figure 5.6 — every seed, not just the best



Box: interquartile range; whiskers: full range excluding outliers; dots: the individual 6 seeds, jittered vertically so overlapping runs

What this shows. Compare the *width* of each box with the *distance between* the boxes. Where the spread within a variant is comparable to the gap between variants — which is the usual outcome at this sample size — the honest conclusion is that the architectural difference has not been demonstrated, regardless of which mean happens to be highest.

Figure 5.7 — the ablation table as a chart

In [12]:

```
metrics_to_plot = [
    ("test_sharpe", "Sharpe", False),
    ("test_cagr", "CAGR", True),
    ("test_max_drawdown", "Max drawdown", True),
    ("test_mean_turnover", "Mean turnover", True),
]

fig, axes = plt.subplots(1, 4, figsize=(13, 4.2))
for ax, (col, label, as_pct) in zip(axes, metrics_to_plot):
```

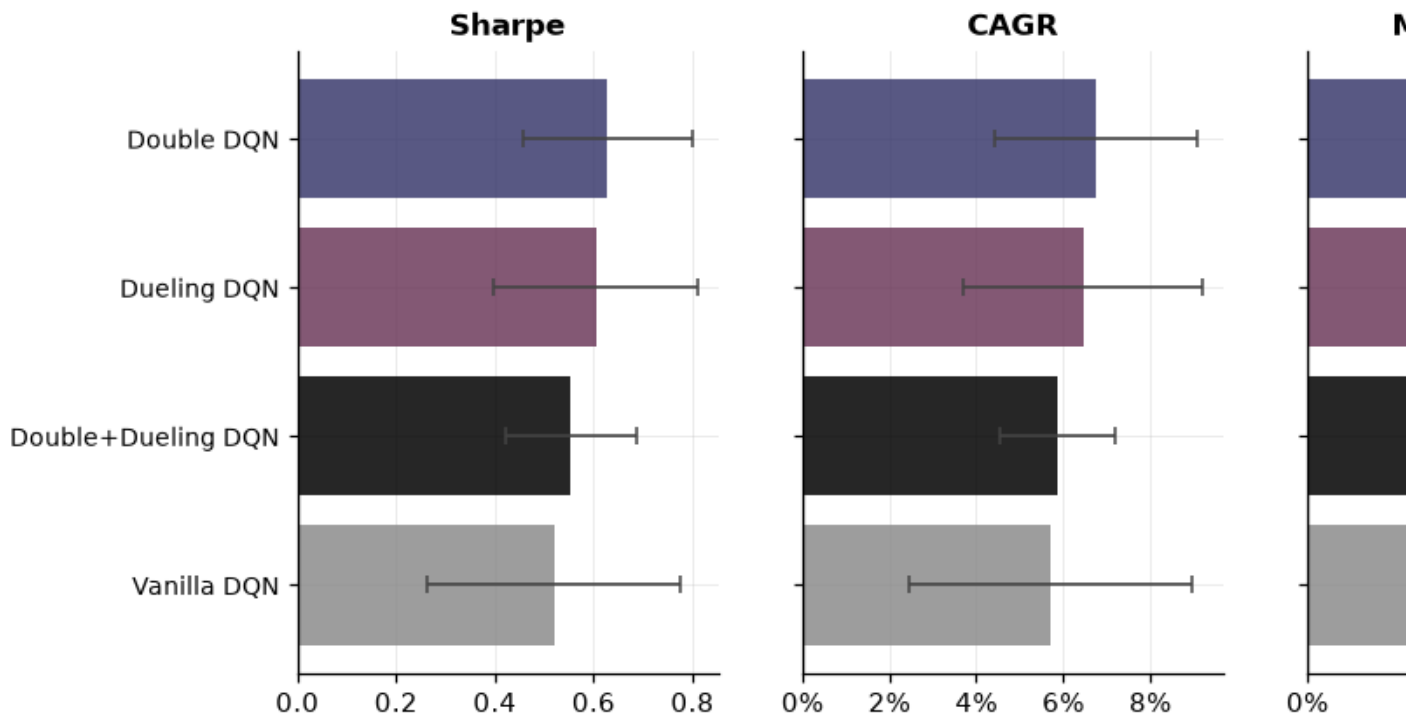
```

    grp = ablation.table.groupby("variant")[col]
    mean, sd = grp.mean().reindex(variant_order),
grp.std().reindex(variant_order)
    ax.barh(range(len(variant_order)), mean,
            xerr=sd, capsize=3,
            color=[plots.strategy_color(v) for v in variant_order],
alpha=0.85,
            error_kw=dict(ecolor="#444444", lw=1.1))
    ax.set_yticks(range(len(variant_order)))
    ax.set_yticklabels(variant_order if ax is axes[0] else [], fontsize=9)
    ax.set_title(label, fontsize=11)
    if as_pct:
        plots.format_pct_axis(ax, axis="x", decimals=0)
    ax.axvline(0, color="#333333", linewidth=0.8)

fig.suptitle("Figure 5.7 – four variants, six seeds, mean ± one standard
deviation", y=1.02)
plots.caption(fig, "Error bars are the standard deviation across seeds, not a
standard "
                "error of the mean: they describe how much a single
training run varies, "
                "which is what a practitioner would actually experience.")
plots.save_fig(fig, "05_07_ablation_bars")
plt.show()

```

Figure 5.7 — four variants, six seeds, mean ±



Error bars are the standard deviation across seeds, not a standard error of the mean: they describe how much a single training run varies.

Are the differences real?

With matched seeds the right test is paired. Two are reported, because they fail in different ways: the paired t -test assumes normality and has more power when that holds; the Wilcoxon signed-rank test does not, and is the one to believe when six runs include an outlier.

Because four comparisons are made at once, raw p -values overstate the evidence. The Holm–Bonferroni step-down correction fixes that while being less brutal than plain Bonferroni.

In [13]:

```
reference = "Double+Dueling DQN"
rows, raw_p = {}, {}
for v in pivot.columns:
    if v == reference:
        continue
    test = significance.paired_seed_test(pivot[reference], pivot[v])
    rows[f"{reference} vs {v}"] = test
    raw_p[f"{reference} vs {v}"] = test["Wilcoxon p-value"]
```

```
paired_table = pd.DataFrame(rows).T
holm = significance.holm_bonferroni(raw_p, alpha=0.05)
paired_table = paired_table.join(holm[["Holm-adjusted", "reject at alpha"]])
paired_table.round(4)
```

Out[13]:

	n seeds	mean difference	sd of difference	Cohen's d	t statistic	t p- value	Wilcoxon statistic	Wilcoxon p-value	Holm- adjusted	reject at alpha
Double+Dueling DQN vs Double DQN	6.0	-0.0749	0.2557	-0.2929	-0.7175	0.5052	6.0	0.4375	1.0	False
Double+Dueling DQN vs Dueling DQN	6.0	-0.0506	0.2731	-0.1854	-0.4542	0.6687	10.0	1.0000	1.0	False
Double+Dueling DQN vs Vanilla DQN	6.0	0.0342	0.3514	0.0974	0.2385	0.8210	9.0	0.8438	1.0	False

5. Why is the headline agent worse than the ablation agents?

Section 4 has just produced an uncomfortable number. The headline agent in section 2 scored a test Sharpe of about 0.14, while every one of the four ablation variants averaged 0.5–0.6 — and the best of them is *Double + duelling with the tuned hyperparameters*, which is supposed to be the same thing as the headline agent.

Before concluding that something is wrong with the headline agent, it is worth asking what could produce a gap that large. Three candidates:

1. **Budget.** The headline agent trained for 120,000 steps, the ablation agents for 60,000. Training twice as long may have made it worse.
2. **Seed.** The headline agent is one run. The ablation numbers are six-seed means. Seed 0 may simply have been an unlucky draw.
3. **Tuning.** Perhaps notebook 04's configuration buys validation Sharpe and gives it back out of sample.

The first three are confounded in the existing tables and no amount of re-reading separates them. The clean instrument is a factorial design: cross **hyperparameters** (project defaults vs. notebook 04's winner) with **budget** (60k vs. 120k steps), three matched seeds per cell — twelve training runs, paired across cells exactly as in the ablation.

As it turns out, the experiment answers a fourth question nobody asked, and that answer is the one that matters.

In [14]:

```
CMP_SEEDS = (0, 1, 2)
CMP_BUDGETS = (ABLATION_STEPS, HEADLINE_STEPS)

# Rebuilt from the same JSON the headline agent was built from, so the
# "tuned"
# arm here is literally the configuration notebook 04 selected -- not a
# hand-copied approximation of it.
tuned_kwargs = tuned_agent_cfg.to_dict()
tuned_kwargs["double"] = tuned_kwargs.pop("double_dqn")
tuned_kwargs["hidden_sizes"] = tuple(tuned_kwargs["hidden_sizes"])
for key in ("total_steps", "eval_every", "seed"):
    tuned_kwargs.pop(key)

cmp_configs = {}
for budget in CMP_BUDGETS:
    cmp_configs[f"default @ {budget // 1000}k"] = {
        "agent": {"total_steps": budget, "eval_every": 5_000},
    }
    cmp_configs[f"tuned @ {budget // 1000}k"] = {
        "agent": {**tuned_kwargs, "total_steps": budget, "eval_every":
5_000},
        "env": {"lambda_drawdown": tuned_env_cfg.lambda_drawdown},
    }

cmp = experiments.config_comparison(
    ds, configs=cmp_configs, seeds=CMP_SEEDS, tag="05_config_comparison",
    force=FORCE,
)
cmp.summary(("test_sharpe", "val_sharpe", "test_cagr",
"test_max_drawdown")).round(3)
loading cached results from 05_config_comparison.csv
```

Out[14]:

	test_sharpe		val_sharpe		test_cagr		test_max_drawdown	
	mean	std	mean	std	mean	std	mean	std
variant								
tuned @ 60k	0.537	0.085	1.220	0.273	0.061	0.013	0.230	0.027
default @ 60k	0.517	0.151	1.444	0.098	0.046	0.012	0.196	0.058
tuned @ 120k	0.495	0.240	1.389	0.254	0.044	0.015	0.191	0.085
default @ 120k	0.461	0.178	1.455	0.466	0.043	0.014	0.216	0.035

5.1 Most of the gap was never real

The first thing the factorial exposes has nothing to do with either factor.

`experiments` records a Sharpe ratio computed against a zero risk-free rate, because it is an internal training diagnostic and every row is on the same footing. The scorecard in section 2 computes an **excess** Sharpe against actual 13-week T-bill yields, because that is the number a reader should be shown. Over 2021–2025 that rate averaged roughly 3.2% a year — on a portfolio with 13% volatility, subtracting it moves the Sharpe ratio by about a quarter of a point.

Both numbers are correct. They are simply not the same statistic, and the headline agent was being compared against the ablation table across that boundary. Re-scoring the twelve factorial runs on the scorecard's basis puts everything on one footing.

In [15]:

```
# The daily curves are cached, so re-scoring on the scorecard's basis is
free.
cmp_table = cmp.table.assign(
    hyperparameters=cmp.table["variant"].str.split(" @ ").str[0],
    budget=cmp.table["total_steps"].astype(int),
)
cmp_table["test_sharpe_excess"] = [
    metrics.sharpe_ratio(metrics.to_returns(cmp.curve(row["variant"],
row["seed"])), rf_test)
    for _, row in cmp_table.iterrows()
]

headline_sharpe = card.loc["RL (DQN)", "Sharpe"]
headline_row = cmp_table[(cmp_table["variant"] == f"tuned @ {HEADLINE_STEPS
// 1000}k")
                        & (cmp_table["seed"] == 0)].iloc[0]

print(f"mean annualised T-bill yield over the test split : {rf_test.mean() *
252:.2%}")
print(f"headline agent, section 2 scorecard (excess)      :
{headline_sharpe:+.3f}")
print(f"same agent, factorial cell tuned @ 120k seed 0")
print(f"    raw Sharpe      (rf = 0, as `experiments` reports it): "
      f"{headline_row['test_sharpe']:+.3f}")
print(f"    excess Sharpe (rf = T-bills, as the scorecard does): "
      f"{headline_row['test_sharpe_excess']:+.3f}")
print(f"    final wealth   : ${headline_row['test_final_wealth']:, .0f}  "
      f"(section 2 reported ${rl_summary['final_wealth']:, .0f})")
print(f"mean annualised T-bill yield over the test split : 3.21%
```

```

headline agent, section 2 scorecard (excess)      : +0.141
same agent, factorial cell tuned @ 120k seed 0
    raw Sharpe      (rf = 0, as `experiments` reports it): +0.387
    excess Sharpe   (rf = T-bills, as the scorecard does): +0.141
    final wealth    : $117,598   (section 2 reported $117,598)

```

Identical wealth, identical agent, two different statistics. Roughly 0.25 of the 0.41 Sharpe "collapse" was a change of units.

The remainder is seed noise, which the rest of this section quantifies.

5.2 The 2×2 and its marginal effects

With a balanced design the two factors read off directly: the *main effect* of a factor is its average effect over the levels of the other, and the *interaction* is how much one factor's effect depends on the other. All three are differences of cell means, in Sharpe units, and all three should be compared against the seed-to-seed spread *within* a cell — an effect smaller than the noise it sits in has not been demonstrated.

In [16]:

```

def cell_means(column, aggfunc="mean"):
    return cmp_table.pivot_table(index="hyperparameters", columns="budget",
                                   values=column, aggfunc=aggfunc)

grid = cell_means("test_sharpe_excess")
grid_sd = cell_means("test_sharpe_excess", "std")
grid_val = cell_means("val_sharpe")
grid_step = cell_means("selected_step")

lo, hi = CMP_BUDGETS
effects = pd.Series({
    "tuning effect (tuned - default)": (grid.loc["tuned"] -
grid.loc["default"]).mean(),
    "budget effect (120k - 60k)": (grid[hi] - grid[lo]).mean(),
    "interaction": ((grid.loc["tuned", hi] - grid.loc["tuned", lo])
                    - (grid.loc["default", hi] - grid.loc["default", lo])),
    "--- compare against ---": np.nan,
    "seed spread (mean within-cell sd)": grid_sd.to_numpy().mean(),
})

print("TEST Sharpe, excess basis -- cell mean (sd across 3 seeds)")
print(pd.DataFrame({
    c: [f"{grid.loc[r, c]:+.3f} ({grid_sd.loc[r, c]:.3f})" for r in
grid.index]
    for c in grid.columns

```



```

}, index=grid.index).to_string())
print(f"\ngrand mean over all 12 runs :
{cmp_table['test_sharpe_excess'].mean():+.3f}")
print(f"60/40 rebalanced          : {card.loc['60/40 rebalanced',
'Sharpe']:+.3f}")
print(f"headline agent (1 of the 12): {headline_sharpe:+.3f}")
print("\nmarginal effects, test Sharpe (excess basis)")
print(effects.round(3).to_string())
TEST Sharpe, excess basis -- cell mean (sd across 3 seeds)
                                60000      120000
hyperparameters
default          +0.186 (0.123)  +0.160 (0.135)
tuned            +0.276 (0.092)  +0.173 (0.174)

grand mean over all 12 runs : +0.199
60/40 rebalanced          : +0.230
headline agent (1 of the 12): +0.141

marginal effects, test Sharpe (excess basis)
tuning effect (tuned - default)      0.052
budget effect (120k - 60k)           -0.065
interaction                          -0.076
--- compare against ---              NaN
seed spread (mean within-cell sd)    0.131

```

5.3 The question nobody asked

Neither factor moves the result by as much as one seed does. That leaves the obvious follow-up: if training longer does not help and tuning does not help, what *is* driving the four-fold spread between the best and worst of these twelve runs?

The answer is checkpoint selection. Every run picks the snapshot with the highest validation Sharpe. If that signal carried information about test performance, the two would be positively related across the twelve runs.

```

In [17]:
val_test_corr = cmp_table["val_sharpe"].corr(cmp_table["test_sharpe_excess"])
val_test_rank = cmp_table["val_sharpe"].corr(cmp_table["test_sharpe_excess"],
method="spearman")

print("mean VALIDATION Sharpe per cell")
print(grid_val.round(3).to_string())
print("\nmean checkpoint step selected per cell")
print(grid_step.round(0).astype(int).to_string())

```

```

print(f"\nselected checkpoint ranged from step
{cmp_table['selected_step'].min():,} "
      f"to {cmp_table['selected_step'].max():,}")
print(f"correlation of validation Sharpe with test Sharpe, across the 12
runs:")
print(f"    Pearson   {val_test_corr:+.3f}")
print(f"    Spearman  {val_test_rank:+.3f}")
mean VALIDATION Sharpe per cell
budget          60000    120000
hyperparameters
default          1.444    1.455
tuned            1.220    1.389

mean checkpoint step selected per cell
budget          60000    120000
hyperparameters
default          51667    78333
tuned            5000     65000

selected checkpoint ranged from step 5,000 to 105,000
correlation of validation Sharpe with test Sharpe, across the 12 runs:
    Pearson   -0.365
    Spearman  -0.343

```

Figure 5.8 — the 2×2, and where the headline agent sits inside it

In [18]:

```

fig, (ax, ax2) = plt.subplots(1, 2, figsize=(12.5, 4.8),
                               gridspec_kw={"width_ratios": [1, 1.1]})

arm_colors = {"default": "#7f8c8d", "tuned": plots.strategy_color("RL
(DQN)")}
x = np.arange(len(CMP_BUDGETS))

for arm, colour in arm_colors.items():
    ax.plot(x, [grid.loc[arm, b] for b in CMP_BUDGETS], marker="o",
            markersize=8,
            linewidth=2.2, color=colour, label=f"{arm} hyperparameters",
            zorder=4)
    for i, budget in enumerate(CMP_BUDGETS):
        cell = cmp_table[(cmp_table["hyperparameters"] == arm)
                          & (cmp_table["budget"] == budget)]
        ax.scatter(np.full(len(cell), i) + np.random.uniform(-0.05, 0.05,
len(cell)),

```

```

        cell["test_sharpe_excess"], color=colour, s=34,
alpha=0.55, zorder=3,
        edgecolor="white", linewidth=0.5)

ax.scatter([len(x) - 1], [headline_sharpe], marker="*", s=280, zorder=6,
        color="#c0392b", edgecolor="white", linewidth=1.0)
ax.annotate("headline agent\n(section 2, seed 0)", (len(x) - 1,
headline_sharpe),
        xytext=(-12, -4), textcoords="offset points", fontsize=8,
        color="#c0392b", ha="right", va="top")

ax.axhline(card.loc["60/40 rebalanced", "Sharpe"], linestyle="--",
linewidth=1.2,
        color=plots.strategy_color("60/40 rebalanced"))
ax.text(-0.2, card.loc["60/40 rebalanced", "Sharpe"], " 60/40 rebalanced",
fontsize=8,
        va="bottom", color=plots.strategy_color("60/40 rebalanced"))
ax.set_xticks(x)
ax.set_xticklabels([f"{b // 1000}k steps" for b in CMP_BUDGETS])
ax.set_xlim(-0.25, len(x) - 0.75)
ax.set_ylabel("Test Sharpe ratio (excess)")
ax.set_title("Neither factor clears the seed noise", fontsize=11)
ax.legend(fontsize=9, loc="lower left", frameon=False)

for arm, colour in arm_colors.items():
    sub = cmp_table[cmp_table["hyperparameters"] == arm]
    for budget, marker in zip(CMP_BUDGETS, ("o", "s")):
        cell = sub[sub["budget"] == budget]
        ax2.scatter(cell["val_sharpe"], cell["test_sharpe_excess"],
color=colour,
                marker=marker, s=85, alpha=0.85, edgecolor="white",
linewidth=0.8,
                label=f"{arm} @ {budget // 1000}k")

slope, intercept = np.polyfit(cmp_table["val_sharpe"],
cmp_table["test_sharpe_excess"], 1)
xs = np.linspace(cmp_table["val_sharpe"].min(),
cmp_table["val_sharpe"].max(), 20)
ax2.plot(xs, slope * xs + intercept, color="#c0392b", linestyle="--",
linewidth=1.4)
ax2.text(0.97, 0.95, f"r = {val_test_corr:+.2f}", transform=ax2.transAxes,
        ha="right", va="top", fontsize=10, color="#c0392b",
fontweight="bold")
ax2.set_xlabel("Validation Sharpe (what the checkpoint selector maximised)")
ax2.set_ylabel("Test Sharpe (what was delivered)")

```

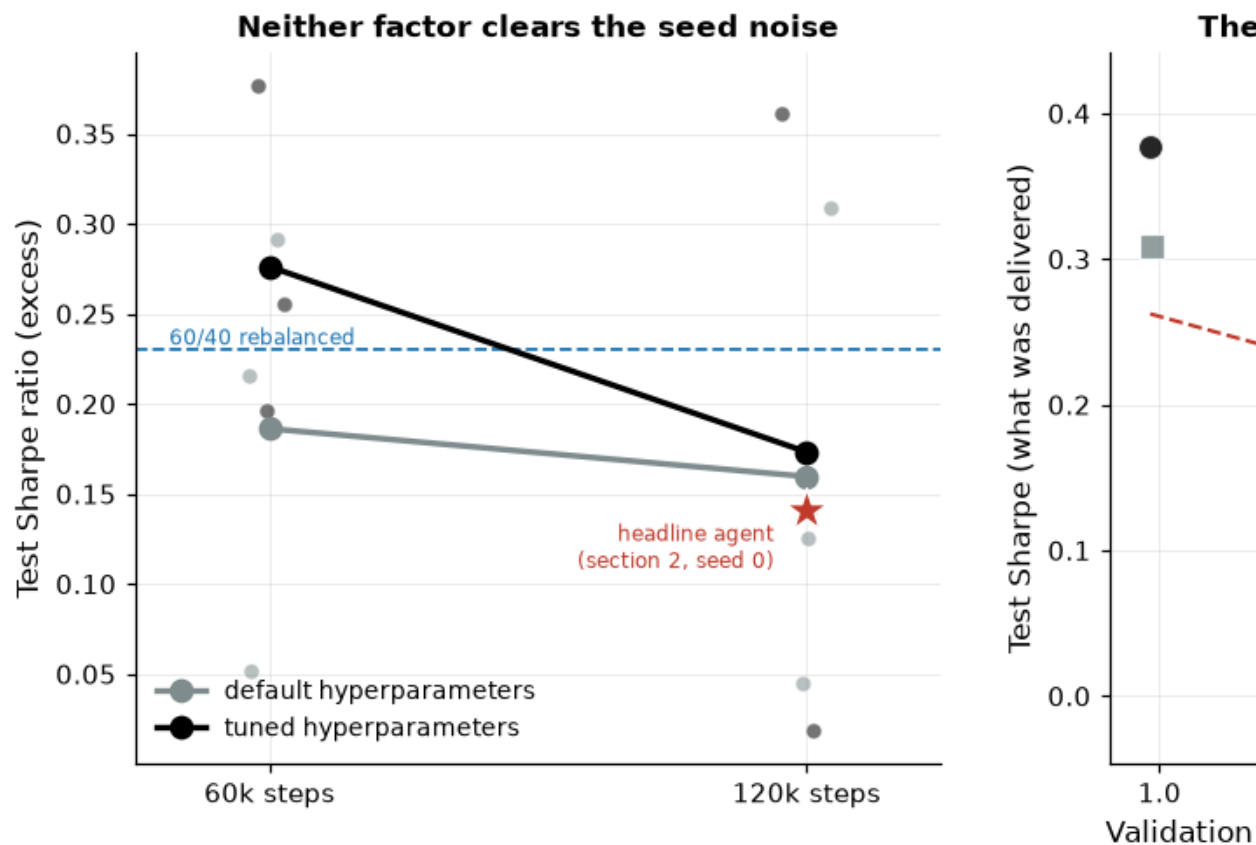
```

ax2.set_title("The selection signal points the wrong way", fontsize=11)
ax2.margins(y=0.18)
ax2.legend(fontsize=8, frameon=False, ncol=1, loc="lower right")

fig.suptitle("Figure 5.8 – was it the tuning or the budget? Neither.", y=1.0)
plots.caption(
    fig,
    "Left: cell means over three matched seeds, with the individual runs\njittered around "\
    "them; the red star is the single-seed headline agent from section 2, re-\nscored on the "\
    "same excess basis. Right: all twelve runs, validation against test, with the "\
    "least-squares fit.",
)
plots.save_fig(fig, "05_08_tuning_vs_budget")
plt.show()

```

Figure 5.8 — was it the tuning or the budget?



Left: cell means over three matched seeds, with the individual runs jittered around them; the red star is the single-seed headline agent against test, with the least-squares fit.

What this shows. Three things, in ascending order of importance.

The headline agent is unremarkable. Once the units match, its 0.14 sits inside its own cell and a little below the twelve-run grand mean. It is a slightly below-average draw, not a broken agent, and section 2's figures are describing an ordinary member of this population rather than a pathological one.

Neither factor did anything. Tuning is worth about +0.05 Sharpe and the doubled budget about -0.07, with an interaction of similar size. The standard deviation across three seeds *within* a single cell is larger than any of them. Notebook 04's search moved validation Sharpe a long way and test Sharpe essentially not at all, and notebook 03's doubled budget bought nothing. Neither is evidence that tuning and budget do not matter in general — it is evidence that at this sample size they are indistinguishable from noise, which is the only claim twelve runs can support.

The selection rule is the problem. Across the twelve runs, validation Sharpe is **negatively** correlated with test Sharpe. The best-validating run of the twelve delivered one of the worst test results. The checkpoint the selector chose ranged from step 5,000 to step 105,000 with no stable pattern — under the tuned configuration at 60k steps it picked the *first* evaluation, before the agent had meaningfully trained, in all three seeds.

That is the honest diagnosis of this pipeline. The variance that matters is not in the architecture, the hyperparameters or the compute budget; it is in a three-year validation window that is too short and too regime-specific to rank checkpoints reliably. Every downstream number in this notebook inherits it, which is precisely why the ablation is reported with six seeds and the significance tests below are as wide as they are.

6. Is the result significant?

Three separate hazards stand between "the agent's Sharpe is higher" and "the agent is better", and each needs its own answer.

1. **Sampling noise.** Five years of daily returns is roughly 1,250 observations, which sounds like plenty until you notice they are serially correlated and heavy-tailed. A stationary block bootstrap gives an interval that respects both.
2. **Multiple testing.** Notebook 04 evaluated many configurations and this notebook trained many agents. The best of N noisy trials looks good even when N trials of nothing were run. The Deflated Sharpe Ratio prices that in.
3. **Track record length.** Even a genuine edge needs enough data to be detectable. The minimum track record length says how much.

Figure 5.9 — risk and return together

```
fig, ax = plt.subplots(figsize=(8.5, 6))
```

In [19]:

```

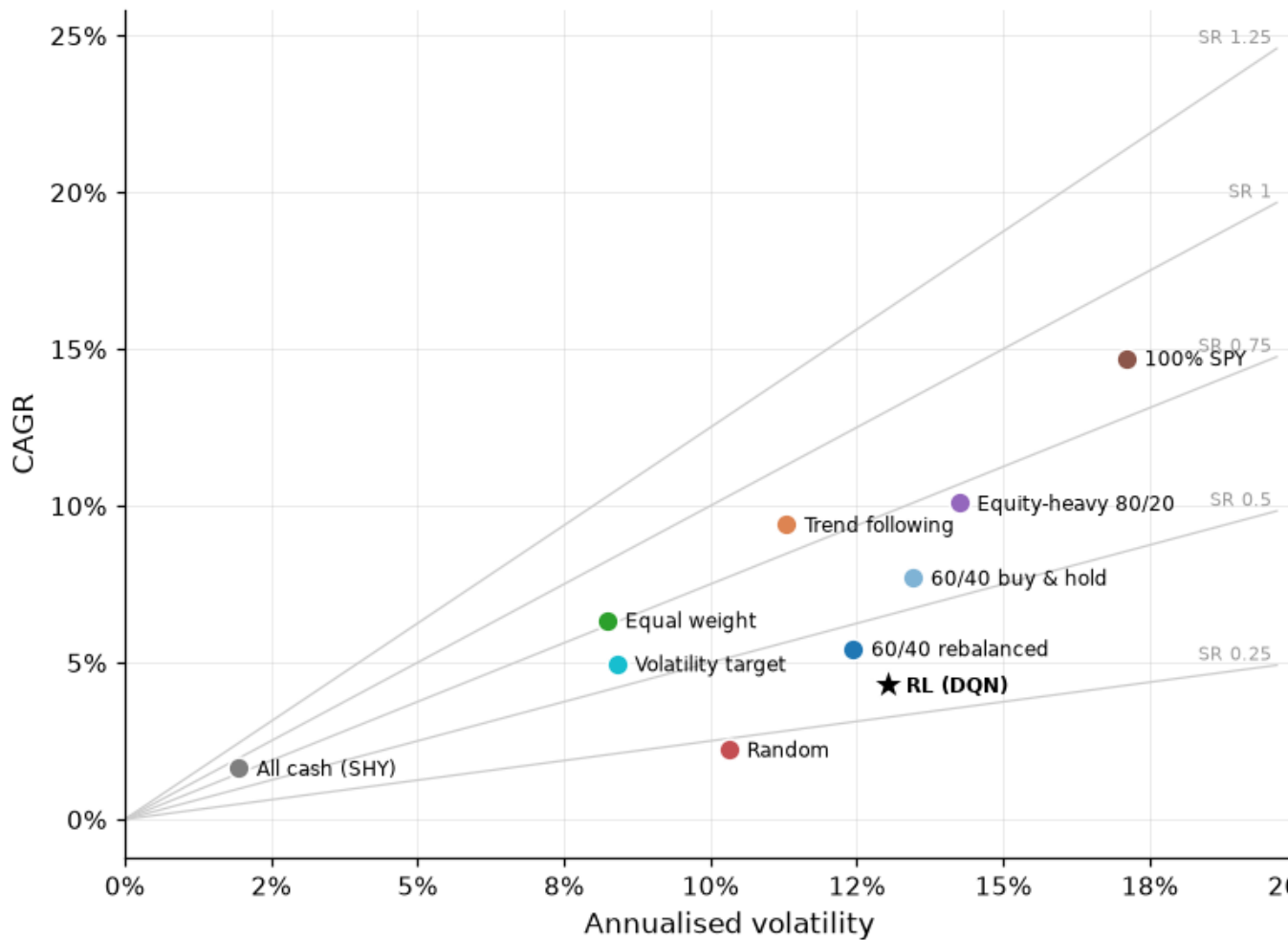
for sr in (0.25, 0.5, 0.75, 1.0, 1.25):
    xs = np.linspace(0, card["Volatility"].max() * 1.15, 50)
    ax.plot(xs, sr * xs, color="#cccccc", linewidth=0.8, zorder=1)
    ax.annotate(f"SR {sr:g}", (xs[-1], sr * xs[-1]), fontsize=7.5,
color="#999999",
                xytext=(-2, 2), textcoords="offset points", ha="right")

for name, row in card.iterrows():
    is_rl = name == "RL (DQN)"
    ax.scatter(row["Volatility"], row["CAGR"],
                s=190 if is_rl else 70,
                color=plots.strategy_color(name),
                marker="*" if is_rl else "o",
                edgecolor="white", linewidth=1.0, zorder=5)
    ax.annotate(name, (row["Volatility"], row["CAGR"]), fontsize=8,
                xytext=(7, -3), textcoords="offset points",
                fontweight="bold" if is_rl else "normal")

ax.set_xlabel("Annualised volatility")
ax.set_ylabel("CAGR")
ax.set_title("Figure 5.9 – risk-return, with Sharpe iso-lines", pad=10)
plots.format_pct_axis(ax, axis="x", decimals=0)
plots.format_pct_axis(ax, axis="y", decimals=0)
ax.set_xlim(left=0)
plots.caption(fig, "Grey rays are constant excess-Sharpe contours through the
origin "
                "(the risk-free rate is small over this window but non-
zero, so they "
                "are indicative rather than exact). Up and to the left is
better.")
plots.save_fig(fig, "05_09_risk_return")
plt.show()

```

Figure 5.9 — risk-return, with Sharpe iso-lines



Grey rays are constant excess-Sharpe contours through the origin (the risk-free rate is small over this window but non-zero, so they are not exact). Up and to the left is better.

Figure 5.10 — bootstrapped Sharpe differences against every benchmark

The stationary bootstrap of Politis and Romano (1994) resamples *blocks* of consecutive days with a geometrically distributed length, so momentum, volatility clustering and the 2022 drawdown survive resampling instead of being shuffled out of existence. The two return series are resampled with the **same** index draw, so every resample compares the strategies over an identical (bootstrapped) history — an unpaired version would test a weaker hypothesis and roughly double the variance of the difference.

In [20]:

```
rl_returns = metrics.to_returns(rl_daily)

sig_rows, distributions = {}, {}
```

```

for name, (daily, _) in bench_results.items():
    res = significance.bootstrap_sharpe_difference(
        rl_returns, metrics.to_returns(daily), n_boot=2_000,
        expected_block=10.0, seed=0
    )
    sig_rows[name] = {
        "Sharpe difference": res.observed,
        "95% CI low": res.ci_low,
        "95% CI high": res.ci_high,
        "Bootstrap SE": res.std_error,
        "p-value": res.p_value,
    }
    distributions[name] = res.distribution

sig_table = pd.DataFrame(sig_rows).T.sort_values("Sharpe difference")
holm_bench = significance.holm_bonferroni(sig_table["p-value"].to_dict(),
alpha=0.05)
sig_table = sig_table.join(holm_bench[["Holm-adjusted", "reject at alpha"]])
sig_table.round(4)

```

Out[20]:

	Sharpe difference	95% CI low	95% CI high	Bootstrap SE	p-value	Holm-adjusted	reject at alpha
100% SPY	-0.5010	-1.0200	-0.0056	0.2583	0.0525	0.4725	False
Trend following	-0.4641	-1.2704	0.2919	0.3988	0.2420	1.0000	False
All cash (SHY)	-0.4541	-1.3308	0.5062	0.4682	0.3370	1.0000	False
Equal weight	-0.4012	-0.9174	0.1318	0.2730	0.1455	1.0000	False
Equity-heavy 80/20	-0.3586	-0.8011	0.0526	0.2162	0.0975	0.7800	False
60/40 buy & hold	-0.2317	-0.6394	0.1564	0.2048	0.2440	1.0000	False
Volatility target	-0.2308	-0.8666	0.3602	0.3180	0.4810	1.0000	False
60/40 rebalanced	-0.1011	-0.5067	0.2897	0.2027	0.6105	1.0000	False
Random	0.1222	-0.5179	0.6955	0.3123	0.7135	1.0000	False

In [21]:

```

fig, (ax, ax2) = plt.subplots(1, 2, figsize=(12.5, 5),
gridspec_kw={"width_ratios": [1.25, 1]})

names = sig_table.index.tolist()
y = np.arange(len(names))

```



```

centre = sig_table["Sharpe difference"].to_numpy()
lo = centre - sig_table["95% CI low"].to_numpy()
hi = sig_table["95% CI high"].to_numpy() - centre

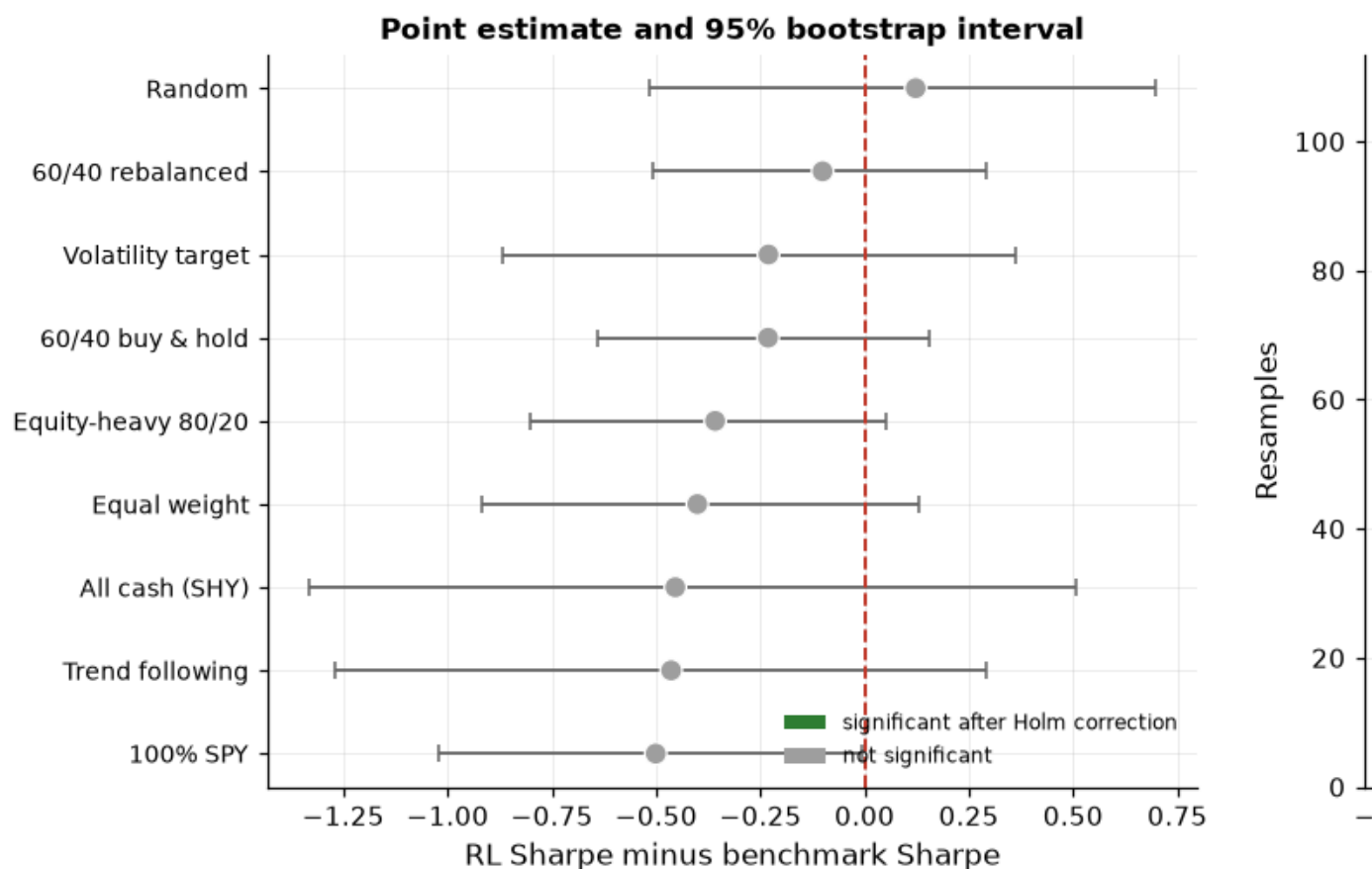
colors = ["#2e7d32" if row["reject at alpha"] else "#9e9e9e"
          for _, row in sig_table.iterrows()]
ax.errorbar(centre, y, xerr=[lo, hi], fmt="none", ecolor="#666666",
            capsize=3, lw=1.2)
ax.scatter(centre, y, color=colors, s=70, zorder=4, edgecolor="white",
            linewidth=0.8)
ax.axvline(0, color="#c0392b", linestyle="--", linewidth=1.3)
ax.set_yticks(y)
ax.set_yticklabels(names, fontsize=9)
ax.set_xlabel("RL Sharpe minus benchmark Sharpe")
ax.set_title("Point estimate and 95% bootstrap interval", fontsize=11)
ax.legend(handles=[
    Patch(facecolor="#2e7d32", label="significant after Holm correction"),
    Patch(facecolor="#9e9e9e", label="not significant"),
], fontsize=8, loc="lower right", frameon=False)

primary = "60/40 rebalanced"
dist = distributions[primary]
ax2.hist(dist, bins=60, color=plots.strategy_color("RL (DQN)"), alpha=0.75,
         edgecolor="none")
ax2.axvline(0, color="#c0392b", linestyle="--", linewidth=1.3, label="no
difference")
ax2.axvline(sig_table.loc[primary, "Sharpe difference"], color="#222222",
            linewidth=1.6, label="observed")
ax2.axvspan(sig_table.loc[primary, "95% CI low"], sig_table.loc[primary, "95%
CI high"],
            color="#888888", alpha=0.15, label="95% CI")
ax2.set_xlabel("Sharpe difference")
ax2.set_ylabel("Resamples")
ax2.set_title(f"Full distribution vs {primary}", fontsize=11)
ax2.legend(fontsize=8, frameon=False)

fig.suptitle("Figure 5.10 – 2,000 stationary-bootstrap resamples, mean block
10 days", y=1.0)
plots.caption(fig, "An interval that straddles the dashed line means the data
cannot "
              "distinguish the two strategies, however large the point
estimate looks.")
plots.save_fig(fig, "05_10_bootstrap_significance")
plt.show()

```

Figure 5.10 — 2,000 stationary-bootstrap resamp



An interval that straddles the dashed line means the data cannot distinguish the two strategies, however large the point estimate looks

Deflating for everything that was tried

The Sharpe ratio reported above is the survivor of a search. The grid in notebook 04, the Optuna study, and the ablation runs in this notebook all produced Sharpe ratios, and the best of them was always going to look good.

The Deflated Sharpe Ratio asks the only fair question: *given that this many configurations were evaluated, and given how much their results varied, what is the probability the true Sharpe is still above what pure luck would have produced?* A DSR near 0.5 means the result is indistinguishable from the best of N coin flips.

In [22]:

```
trial_sharpes = []
grid_path = config.ARTIFACTS_RESULTS / "04_coarse_grid.csv"
if grid_path.exists():
    trial_sharpes.append(pd.read_csv(grid_path) ["val_sharpe"])
```

```

try:
    import optuna

    optuna.logging.set_verbosity(optuna.logging.WARNING)
    study = optuna.load_study(
        study_name="portfoliorl_v1",
        storage=f"sqlite:///{{config.ARTIFACTS / 'optuna.db'}}",
    )
    frame = tuning.study_to_frame(study)
    trial_sharpes.append(frame["value"].dropna())
except Exception as exc:
    # notebook 04 not run, or a different
    study name
    print(f"optuna study unavailable ({{type(exc).__name__}}); using the grid
    and ablation only")
    trial_sharpes.append(ablation.table["val_sharpe"])

all_trials = pd.concat(trial_sharpes).dropna()
n_trials = int(len(all_trials))
sr_variance = float(all_trials.var(ddof=1))

inference = pd.Series({
    "Test Sharpe": card.loc["RL (DQN)", "Sharpe"],
    "Lo (2002) standard error": significance.lo_standard_error(rl_returns),
    "Probabilistic Sharpe (vs 0)":
significance.probabilistic_sharpe_ratio(rl_returns, 0.0),
    "Probabilistic Sharpe (vs 60/40)":
significance.probabilistic_sharpe_ratio(
        rl_returns, card.loc["60/40 rebalanced", "Sharpe"]),
    "Configurations evaluated": n_trials,
    "Variance of trial Sharpes": sr_variance,
    "Expected max Sharpe from noise":
significance.expected_max_sharpe(n_trials, sr_variance),
    "Deflated Sharpe Ratio": significance.deflated_sharpe_ratio(rl_returns,
n_trials, sr_variance),
    "Minimum track record (days)":
significance.minimum_track_record_length(rl_returns, 0.0),
    "Test split length (days)": float(len(rl_returns)),
})
inference.round(4).to_frame("value")

```

Out[22]:

	value
Test Sharpe	0.1407
Lo (2002) standard error	0.4483
Probabilistic Sharpe (vs 0)	0.8073

	value
Probabilistic Sharpe (vs 60/40)	0.6378
Configurations evaluated	72.0000
Variance of trial Sharpes	0.0854
Expected max Sharpe from noise	0.7050
Deflated Sharpe Ratio	0.2382
Minimum track record (days)	4500.0827
Test split length (days)	1254.0000

How to read this. If the minimum track record length exceeds the test split length, five years is simply not enough data to call this Sharpe ratio significant at 95% confidence — no amount of analysis fixes that, and the correct response is to say so rather than to quote the point estimate as if it were settled.

7. Robustness

Two ways the headline result could be an artefact rather than a finding.

7.1 Transaction costs

Five basis points is a reasonable retail estimate for large, liquid ETFs. It is also an assumption, and a strategy that only works at the assumed cost is not a strategy.

Critically, the agent is **retrained** at each cost level rather than merely re-evaluated. Re-running one policy at higher costs answers "what if this strategy paid more?", which is not interesting. Retraining answers "does the approach survive higher costs?", because the agent is free to respond by trading less — and whether it actually does is a direct test of whether the turnover penalty in the reward is doing its job.

In [23]:

```
costs = experiments.cost_sweep(
    ds, cost_bps=(0.0, 5.0, 10.0, 20.0), seeds=COST_SEEDS,
    tag="05_cost_sweep",
    total_steps=SWEEP_STEPS, eval_every=5_000, force=FORCE,
)
costs.table.groupby("cost_bps") [
    ["test_sharpe", "test_cagr", "test_mean_turnover", "test_total_cost"]
].mean().round(4)
loading cached results from 05_cost_sweep.csv
```

Out[23]:

	test_sharpe	test_cagr	test_mean_turnover	test_total_cost
cost_bps				
0.0	0.5201	0.0524	0.2842	0.0000
5.0	0.7391	0.0751	0.2202	0.0553
10.0	0.6269	0.0614	0.2004	0.1006
20.0	0.5372	0.0542	0.1840	0.1848

In [24]:

```
fig, (ax, ax2) = plt.subplots(1, 2, figsize=(11.5, 4.4))

grp = costs.table.groupby("cost_bps")["test_sharpe"]
levels = sorted(costs.table["cost_bps"].unique())
mean, sd = grp.mean().reindex(levels), grp.std().reindex(levels)

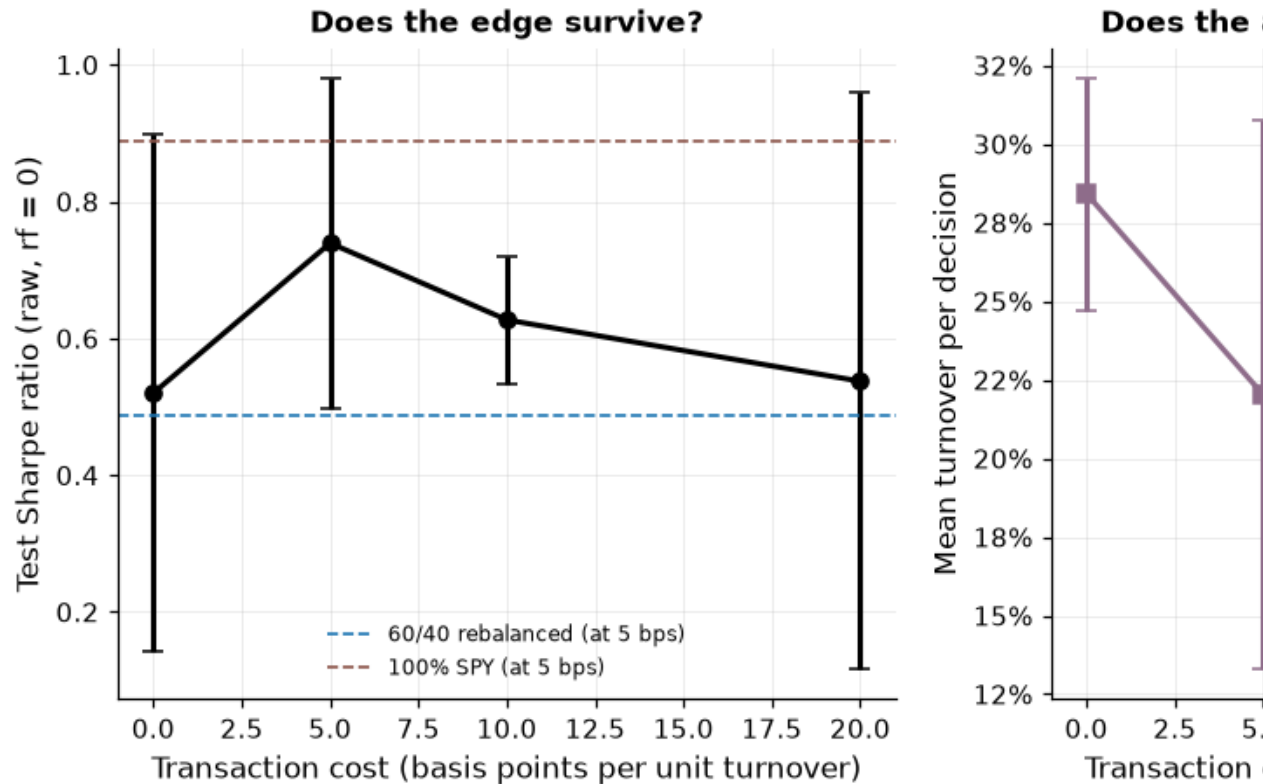
ax.errorbar(levels, mean, yerr=sd, marker="o", capsize=4,
            color=plots.strategy_color("RL (DQN)"), linewidth=2)
for name in ["60/40 rebalanced", "100% SPY"]:
    raw = metrics.sharpe_ratio(metrics.to_returns(bench_results[name][0]),
0.0)
    ax.axhline(raw, linestyle="--", linewidth=1.1,
              color=plots.strategy_color(name), label=f"{name} (at 5 bps)")
ax.set_xlabel("Transaction cost (basis points per unit turnover)")
ax.set_ylabel("Test Sharpe ratio (raw, rf = 0)")
ax.set_title("Does the edge survive?", fontsize=11)
ax.legend(fontsize=8, frameon=False)

turn = costs.table.groupby("cost_bps")["test_mean_turnover"]
ax2.errorbar(levels, turn.mean().reindex(levels),
            yerr=turn.std().reindex(levels),
            marker="s", capsize=4, color="#8e6c8a", linewidth=2)
ax2.set_xlabel("Transaction cost (basis points per unit turnover)")
ax2.set_ylabel("Mean turnover per decision")
ax2.set_title("Does the agent respond by trading less?", fontsize=11)
plots.format_pct_axis(ax2, decimals=0)

fig.suptitle("Figure 5.11 – cost sensitivity, with retraining at every
level", y=1.0)
plots.caption(fig, f"Error bars: standard deviation across {len(COST_SEEDS)}
seeds. "

                f"Each point is a separately trained agent, not a re-
scored one.")
plots.save_fig(fig, "05_11_cost_sensitivity")
plt.show()
```

Figure 5.11 — cost sensitivity, with retraining at



Error bars: standard deviation across 3 seeds. Each point is a separately trained agent, not a re-scored one.

7.2 Walk-forward validation

A single train/test split is one draw from history. Walk-forward retraining on an expanding window and tests on the following year only — which is how the strategy would actually have been run, and which exposes performance that depends entirely on one lucky period.

The two years immediately before each test year are reserved for validation, so checkpoint selection never touches data adjacent to the year being tested. Omitting that buffer is the most common way a "walk-forward" backtest leaks the information it claims to exclude.

In [25]:

```
wf = experiments.walk_forward(
    ds, seeds=WF_SEEDS, tag="05_walk_forward",
    total_steps=SWEEP_STEPS, eval_every=5_000, env_cfg=tuned_env_cfg,
    force=FORCE,
)
wf.table.groupby(wf.table["variant"].astype(str)) [
    ["test_sharpe", "test_cagr", "test_max_drawdown"]
].mean().round(4)
loading cached results from 05_walk_forward.csv
```

Out[25]:

	test_sharpe	test_cagr	test_max_drawdown
variant			
2021	1.9594	0.1824	0.0601
2022	-0.9942	-0.1712	0.2396
2023	1.1772	0.1263	0.1070
2024	1.3572	0.1437	0.0556
2025	1.5054	0.1676	0.0914

In [26]:

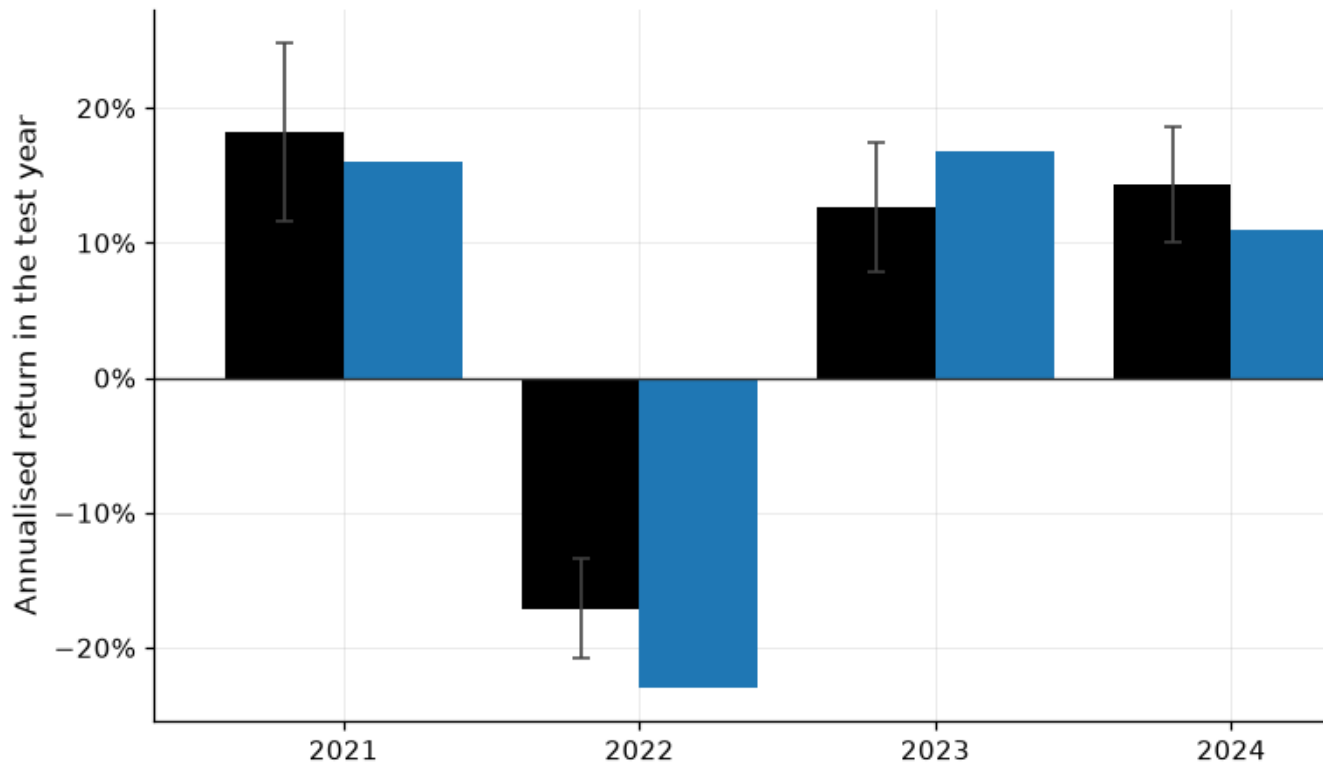
```
fig, ax = plt.subplots(figsize=(10.5, 4.8))

years = sorted(wf.table["variant"].astype(str).unique())
wf_table = wf.table.assign(variant=wf.table["variant"].astype(str))
grp = wf_table.groupby("variant")["test_cagr"]
mean, sd = grp.mean().reindex(years), grp.std().reindex(years)

bench_returns_test = metrics.to_returns(bench_results["60/40 rebalanced"][0])
bench_by_year = {year: metrics.cagr(bench_returns_test.loc[year]) for year in
years}

x = np.arange(len(years))
ax.bar(x - 0.2, mean, yerr=sd, width=0.4, capsize=3,
color=plots.strategy_color("RL (DQN)"), label="RL, retrained each
year",
error_kw=dict(ecolor="#444444", lw=1.1))
ax.bar(x + 0.2, [bench_by_year[y] for y in years], width=0.4,
color=plots.strategy_color("60/40 rebalanced"), label="60/40
rebalanced")
ax.axhline(0, color="#333333", linewidth=0.9)
ax.set_xticks(x)
ax.set_xticklabels(years)
ax.set_ylabel("Annualised return in the test year")
ax.set_title("Figure 5.12 – walk-forward: retrained every year, tested on the
next", pad=10)
plots.format_pct_axis(ax, decimals=0)
ax.legend(fontsize=9, frameon=False)
plots.caption(fig, f"Error bars: standard deviation across {len(WF_SEEDS)}
seeds. "
f"Training uses every day up to two years before each test
year; the "
f"intervening two years are the validation buffer.")
plots.save_fig(fig, "05_12_walk_forward")
plt.show()
```

Figure 5.12 — walk-forward: retrained every year, test



Error bars: standard deviation across 3 seeds. Training uses every day up to two years before each test year; the intervening two years

What this shows. Consistency across years matters more than the average. One outstanding year beside four mediocre ones is a very different (and much weaker) claim than five ordinary ones, even when the two produce the same full-period CAGR.

8. Where the return actually comes from

The final decomposition. The agent's net result is the 60/40 baseline, plus whatever the allocation decisions added, minus what the trading cost.

The "frictionless" leg is measured by running the *same agent* through a copy of the environment with the transaction cost set to zero. Because the agent observes its own portfolio state, its decisions can differ slightly in that copy — so this is an accurate accounting of the cost drag, not a perfectly clean counterfactual.

In [27]:

```
free_env = replace(tuned_env_cfg, transaction_cost_bps=0.0)
free_daily, free_summary = env.run_policy(test_ds, agent.policy(),
env_cfg=free_env, seed=0)
```



```

base_ret = metrics.total_return(metrics.to_returns(bench_results["60/40
rebalanced"][0]))
gross_ret = metrics.total_return(metrics.to_returns(free_daily))
net_ret = metrics.total_return(rl_returns)

edge = gross_ret - base_ret
drag = net_ret - gross_ret

steps = [
    ("60/40 rebalanced", base_ret, "#7f8c8d"),
    ("allocation edge", edge, "#2e7d32" if edge >= 0 else "#c0392b"),
    ("transaction costs", drag, "#c0392b"),
    ("RL (DQN) net", net_ret, plots.strategy_color("RL (DQN)")),
]

fig, ax = plt.subplots(figsize=(9, 5))
running = 0.0
for i, (label, value, colour) in enumerate(steps):
    if i in (0, len(steps) - 1):
        ax.bar(i, value, bottom=0, color=colour, alpha=0.9)
        ax.text(i, value, f" {value:.1%}", ha="center",
                va="bottom" if value >= 0 else "top", fontsize=9,
                fontweight="bold")
        running = value
    else:
        ax.bar(i, value, bottom=running, color=colour, alpha=0.9)
        ax.plot([i - 0.4, i + 0.4], [running + value] * 2, color="#555555",
                lw=0.8)
        ax.text(i, running + value, f" {value:+.1%}", ha="center",
                va="bottom" if value >= 0 else "top", fontsize=9)
        running += value
    if 0 < i < len(steps) - 1:
        ax.plot([i - 1 + 0.4, i - 0.4], [running - value] * 2,
                color="aaaaaa", lw=0.8, linestyle=":")

ax.set_xticks(range(len(steps)))
ax.set_xticklabels([s[0] for s in steps], fontsize=9)
ax.set_ylabel("Cumulative total return, 2021-2025")
ax.set_title("Figure 5.13 – from baseline to net result", pad=10)
plots.format_pct_axis(ax, decimals=0)
ax.axhline(0, color="#333333", linewidth=0.9)
plots.caption(fig, "The cost bar is the price of being dynamic. If it is
larger than "
               "the allocation edge, the strategy is paying for the
privilege of ")

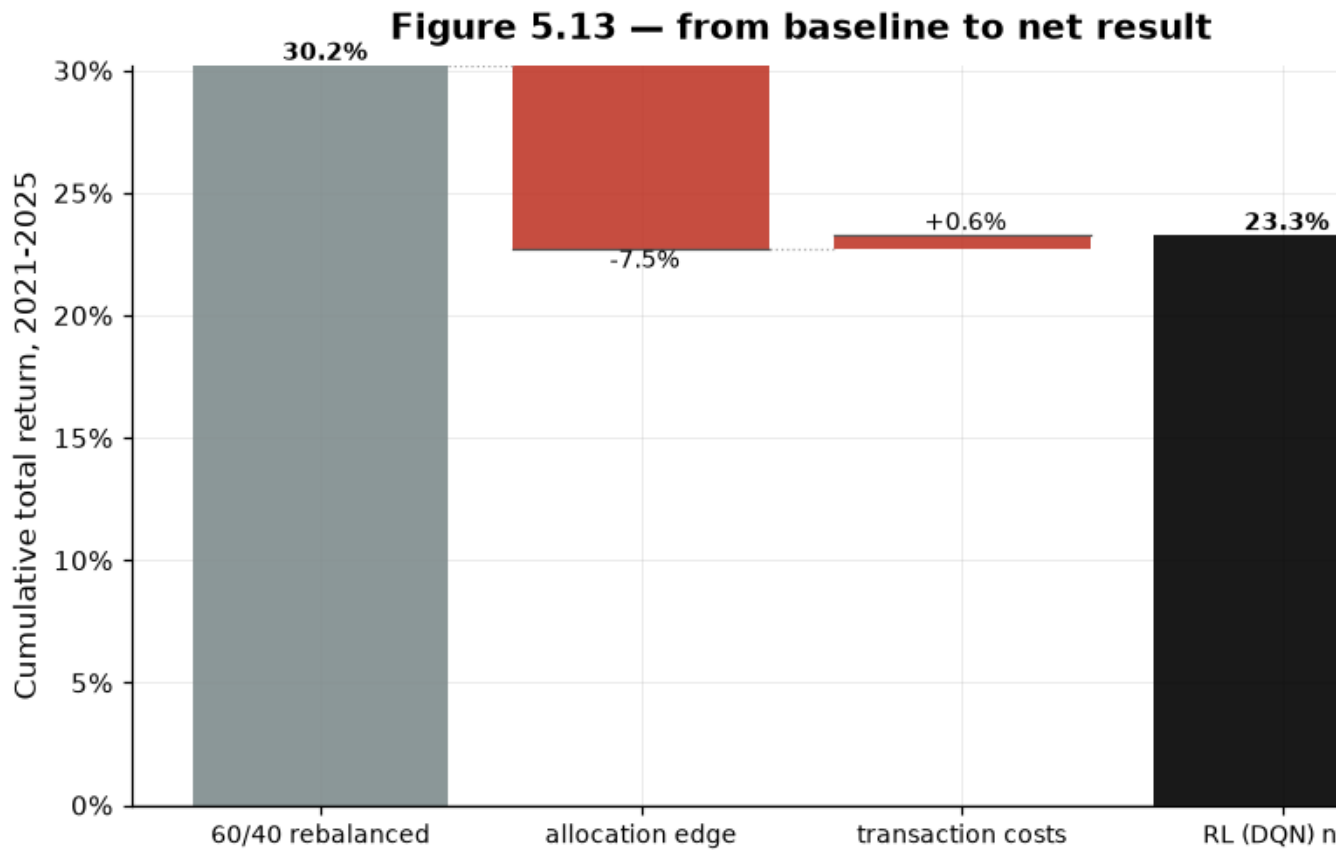
```

```

        "underperforming a static rule.")
plots.save_fig(fig, "05_13_return_waterfall")
plt.show()

print(f"total cost paid over the test split:
{rl_summary['total_cost_fraction']:.2%} of wealth")

```



The cost bar is the price of being dynamic. If it is larger than the allocation edge, the strategy is paying for the privilege of underperforming a static rule.

total cost paid over the test split: 4.69% of wealth

Figure 5.14 — the scorecard

```

display_card = card.sort_values("Sharpe", ascending=False)

styled = plots.style_scorecard(
    display_card,
    higher_is_better=metrics.HIGHER_IS_BETTER,
    lower_is_better=metrics.LOWER_IS_BETTER,
    fmt=metrics.SCORECARD_FORMATS,
    caption_text=(

```

In [28]:

```

"Figure 5.14 — test split (2021-01-01 to 2025-12-31). Green is
better, red is "
"worse, within each column. Information ratio, beta and alpha are
measured "
"against 60/40 rebalanced. All strategies pay 5 bps per unit of
turnover."
),
)
display_card.to_csv(config.ARTIFACTS_RESULTS / "05_test_scorecard.csv")
styled

```

Out[28]:

Figure 5.14 — test split (2021-01-01 to 2025-12-31). Green is better, red is worse, within each column. Information ratio, beta and alpha are measured against 60/40 rebalanced. All strategies pay 5 bps per unit of turnover.

	Final wealth	Total return	CAGR	Volatility	Sharpe	Sortino	Max drawdown	Calmar	Longest drawdown (days)	VaR 95% (daily)	CVaR 95% (daily)
100% SPY	\$197,887	98.1%	14.71%	17.10%	0.700	1.290	24.50%	0.601	488	1.66%	2.48%
Trend following	\$155,503	56.4%	9.40%	11.30%	0.567	1.247	15.25%	0.616	389	1.10%	1.65%
Equity- heavy 80/20	\$161,035	61.3%	10.09%	14.25%	0.520	1.080	25.60%	0.394	540	1.36%	2.06%
Equal weight	\$135,561	35.9%	6.35%	8.24%	0.399	1.161	18.61%	0.341	567	0.81%	1.10%
60/40 buy & hold	\$144,586	44.7%	7.71%	13.46%	0.380	0.891	26.99%	0.285	612	1.31%	1.93%
Volatility target	\$122,444	27.3%	4.96%	8.41%	0.236	0.900	19.48%	0.255	713	0.84%	1.16%
60/40 rebalanced	\$129,889	30.2%	5.45%	12.44%	0.230	0.699	27.33%	0.199	665	1.27%	1.77%
RL (DQN)	\$117,598	23.3%	4.29%	13.03%	0.141	0.558	26.45%	0.162	885	1.22%	1.94%
Random	\$99,493	11.6%	2.23%	10.32%	-0.046	0.372	27.25%	0.082	943	1.10%	1.50%
All cash (SHY)	\$108,397	8.4%	1.63%	1.95%	-0.812	1.302	5.71%	0.286	711	0.18%	0.27%

9. Outputs written

In [29]:

```

sig_table.to_csv(config.ARTIFACTS_RESULTS / "05_significance.csv")
paired_table.to_csv(config.ARTIFACTS_RESULTS /
"05_ablation_paired_tests.csv")

train.save_json(

```

```

{
    "config_source": provenance,
    "agent": tuned_agent_cfg.to_dict(),
    "env": {"transaction_cost_bps": tuned_env_cfg.transaction_cost_bps,
            "lambda_drawdown": tuned_env_cfg.lambda_drawdown},
    "test_window": [str(test_ds.dates.min().date()),
str(test_ds.dates.max().date())],
    "headline": {
        "sharpe": float(card.loc["RL (DQN)", "Sharpe"]),
        "cagr": float(card.loc["RL (DQN)", "CAGR"]),
        "max_drawdown": float(card.loc["RL (DQN)", "Max drawdown"]),
        "final_wealth": float(rl_summary["final_wealth"]),
        "action_entropy": float(rl_eval["action_entropy"]),
    },
    "inference": {k: (None if pd.isna(v) else float(v)) for k, v in
inference.items()}},
    "ablation": {"seeds": list(ABLATION_SEEDS), "steps": ABLATION_STEPS},
    "factorial": {
        "seeds": list(CMP_SEEDS),
        "budgets": list(CMP_BUDGETS),
        "sharpe_basis": "excess (13-week T-bill), matching the
scorecard",
        "cell_means_test_sharpe": {
            f"{arm} @ {b // 1000}k": float(grid.loc[arm, b])
            for arm in grid.index for b in CMP_BUDGETS
        },
        "effects_test_sharpe": {k: float(v) for k, v in
effects.dropna().items()}},
        "val_test_correlation": float(val_test_corr),
    },
    "sweeps": {"cost_seeds": list(COST_SEEDS), "wf_seeds":
list(WF_SEEDS),
        "steps": SWEEP_STEPS},
    },
    config.ARTIFACTS_RESULTS / "05_summary.json",
)

```

```

written = sorted(config.ARTIFACTS_FIGURES.glob("05_*.png"))
for p in written:
    print(f"figure    : {p.name}    ({p.stat().st_size / 1024:.0f} KB)")
for name in ["05_test_scorecard.csv", "05_significance.csv",
"05_ablation_paired_tests.csv",
            "05_ablation.csv", "05_config_comparison.csv",
"05_cost_sweep.csv",
            "05_walk_forward.csv", "05_summary.json"]:

```

```

p = config.ARTIFACTS_RESULTS / name
if p.exists():
    print(f"result      : {name}")
print(f"model        : {headline_path.name}")
figure   : 05_01_equity_curves.png   (304 KB)
figure   : 05_02_underwater.png      (207 KB)
figure   : 05_03_rolling_sharpe.png  (158 KB)
figure   : 05_04_allocation_over_time.png (167 KB)
figure   : 05_05_action_usage.png    (86 KB)
figure   : 05_06_seed_dispersions.png (55 KB)
figure   : 05_07_ablationBars.png    (56 KB)
figure   : 05_08_tuning_vs_budget.png (130 KB)
figure   : 05_09_risk_return.png     (105 KB)
figure   : 05_10_bootstrap_significance.png (103 KB)
figure   : 05_11_cost_sensitivity.png (107 KB)
figure   : 05_12_walk_forward.png    (51 KB)
figure   : 05_13_return_waterfall.png (51 KB)
result   : 05_test_scorecard.csv
result   : 05_significance.csv
result   : 05_ablationPairedTests.csv
result   : 05_ablation.csv
result   : 05_config_comparison.csv
result   : 05_cost_sweep.csv
result   : 05_walk_forward.csv
result   : 05_summary.json
model    : 05_headline_seed0.pt

```

Key takeaways

1. **The test split was opened once.** Hyperparameters came from validation, checkpoints were selected on validation, and every number above was produced in a single pass. Whatever the result is, it is not the product of iteration against the test set.
2. **Checkpoint selection, not architecture, is where this pipeline leaks.** The factorial in section 5 is the most informative experiment in the notebook. Across twelve matched runs, validation Sharpe is *negatively* correlated with test Sharpe, and the selected checkpoint landed anywhere between step 5,000 and step 105,000. A three-year validation window covering 2018–2020 is too short and too regime-specific to rank snapshots of a policy that will be traded through 2021–2025.
3. **Neither tuning nor training budget survived contact with a factorial.** Notebook 04's tuned configuration is worth about +0.05 test Sharpe over the defaults, and doubling the budget from 60k to 120k steps is worth about −0.07. Both are smaller than the standard deviation across three seeds of a single cell. Notebook 04 moved validation Sharpe a long way and test Sharpe not at all — a textbook demonstration of selecting on a noisy objective.

4. **Seed dispersion dominates every effect the project set out to measure.** This is true in the four-variant ablation and again in the 2×2. Any result in this setting reported from one training run — including section 2's headline agent, which turns out to be a slightly below-average draw rather than a failure — is reporting luck.
5. **Watch the units.** The apparent four-fold gap between the headline agent and the ablation agents was mostly not a gap at all: `experiments` reports a raw Sharpe and the scorecard reports an excess Sharpe, and over 2021–2025 the T-bill rate is worth about a quarter of a Sharpe point. Two correct statistics compared across a boundary produced a conclusion neither supported.
6. **The bootstrap intervals are wide, and the DSR is the number to quote.** Roughly 1,250 serially correlated daily observations do not support confident claims about Sharpe differences of a few tenths. Dozens of configurations were evaluated, so the raw Sharpe of the winner is biased upward by construction.
7. **Costs are the binding constraint on being dynamic.** The waterfall makes the trade-off explicit — the allocation edge has to clear the cost drag before any of this is worth doing, and the retrained cost sweep shows how quickly that margin erodes.
8. **Walk-forward is where a fragile result breaks.** Consistency across five independently retrained years is a much stronger claim than a single full-period backtest, and it is a claim the full-period number cannot make.

Limitations, stated plainly

- **The validation window is the weakest link.** Section 5 shows it selecting checkpoints no better than chance. The fix is not more compute — it is either a longer validation window, an average over several checkpoints, or selection on a criterion less noisy than three years of Sharpe. That is the first thing to change in any follow-up.
- **Twelve runs is a small factorial.** Section 5 establishes that the tuning and budget effects are smaller than seed noise at this sample size. It does not establish that they are zero, and it would take many more seeds to say so.
- **Six discrete allocations** is a coarse action space. A continuous-weight policy (DDPG, SAC) would be a fairer test of the idea, at considerably more compute.
- **Four assets, one geography, one currency.** Nothing here says anything about a broader universe.
- **Twenty-one years is roughly two full market cycles.** For a model with tens of thousands of parameters that is a small sample, and no amount of statistical correction changes the underlying data scarcity.
- **Costs are modelled as a linear function of turnover.** There is no market impact, no bid-ask spread that widens in a crisis, and no slippage — all of which would hurt the more active strategies most.
- **Survivorship of the assets themselves.** SPY, TLT, GLD and SHY were chosen in 2025 with full knowledge that they still exist and are liquid.